



# Magnetar: Per-Validator Standalone FIPS 205 SLH-DSA for Lux Public-BFT Consensus

---

Zach Kelling

*Lux Industries*

2026Artefact: `luxfi/magnetar v0.5.2 (commit 1c84519)`

## Abstract

We specify *Magnetar*, the hash-based post-quantum signature primitive of the Lux Polaris cert profile. Magnetar’s headline primitive is a *per-validator standalone* construction: each validator  $i$  holds an independent FIPS 205 SLH-DSA keypair  $(sk_i, pk_i)$  [44], produces a single-party deterministic signature  $\sigma_i$  on the consensus message, and the consensus layer collects  $N$  signatures into a *ValidatorAggregateCert* that verifies under any unmodified FIPS 205 verifier. There is no distributed key generation, no dealer, no aggregator, and no threshold reconstruction step. Each  $\sigma_i$  is byte-identical to what a single-party FIPS 205 SIGNDETERMINISTIC would produce on  $(sk_i, \mu, ctx)$  — this is the *byte-identity claim* pinned by `TESTMAGNETAR_WIRE_FIPS205VERIFIABLE` (`ref/go/pkg/magnetar/wire_test.go:166–220`), which routes through the `ValidatorSign` primary primitive, strips the canonical MAGS/MAGG wire envelope to recover the raw FIPS 205 payload, and feeds it to `cloudflare/circl/sign/slhd` [19] with no Magnetar code path on the verifier side. All three SHAKE parameter sets (SHAKE-192s, SHAKE-192f, SHAKE-256s) pass under `-count=1` and `-race`.

**Why per-validator and not threshold.** SLH-DSA is hash-based and admits no FROST-style linear share-aggregation identity [21, 8, 44]. Cozzo-Smart (EUROCRYPT 2019) and Bonte-Smart-Tan (2023) establish that every internal SHAKE/SHA-2 evaluation in an HBS scheme is a non-linear function of the secret seed; producing a single FIPS 205-shaped signature from  $t$  shares requires either reconstructing the seed (a dealer or aggregator in the trusted computing base) or running a full multi-party computation over the SHA-256/SHAKE hash tree (open research, on the order of  $10^4$  hash invocations per signature under MPC). For *public-BFT* consensus the architecturally correct primitive is therefore  $N$  standalone signatures collected at the consensus layer; the per-validator-standalone construction shipped here is the canonical hash-based instantiation of that pattern.

**Composition.** The  $N \times |\sigma|$  on-wire cost of a  $K = 21$  Lux Polaris quorum at SHAKE-192s is  $\sim 332$  KiB. The Z-Chain Groth16 rollup compresses this to  $\sim 192$  B before the certificate hits long-term storage. The rollup is a separate primitive (lux-zchain spec); the Magnetar wire codec presents the parallel-slices (`Signers`, `PubKeys`, `Sigs`) shape that maps directly onto the rollup witness layout.

The companion artefact is `luxfi/magnetar` v0.5.2 [37], with the wire codec MAGS/MAGG and the stateless `VerifyBytes` dispatcher landing in commit `1c84519` on 2026-05-31. The reference implementation is pure Go, backed by `cloudflare/circl/sign/slhd` v1.6.3 [19] for the underlying FIPS 205 primitive, with the same constant-time posture the `circl` maintainers ship. Magnetar joins Pulsar (Module-LWE) [28] and Corona (Ring-LWE) [30] as the three independent legs of the Lux Polaris maximum-assurance post-quantum cert profile; the three primitives draw on algorithmically distinct families (M-LWE / R-LWE / hash-based) so no single algorithmic break collapses the conjunction.

**Threshold-form availability.** The threshold form of Magnetar — Shamir-VSS over the SLH-DSA scheme seed with reveal-and-aggregate semantics, and a true threshold-HBS construction (THBS) per McGrew et al. ePrint 2019/793 with selected-element reconstruction — ships in the same artefact under `ref/go/pkg/thbs/`. Both forms are *TEE-required by policy*: the dealer (THBS v1) or aggregator (reveal-and-aggregate) holds master secret material in memory and must run inside hardware-attested confidential compute (AMD SEV-SNP, Intel TDX, or equivalent). The threshold forms are appropriate for M-Chain bridge custody and A-Chain confidential compute, where a TEE-attested host is in the TCB by deployment policy. They are explicitly NOT public-BFT-safe and are documented in Appendix A for completeness; production public-consensus deployments use the per-validator standalone primitive of this paper’s main body.

## Contents

|          |                     |          |
|----------|---------------------|----------|
| <b>1</b> | <b>Introduction</b> | <b>5</b> |
| 1.1      | What Magnetar is    | 5        |
| 1.2      | Why hash-based      | 6        |
| 1.3      | What is novel       | 7        |
| 1.4      | Trust model summary | 8        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 1.5      | What this paper specifies . . . . .                                | 8         |
| <b>2</b> | <b>Preliminaries</b>                                               | <b>9</b>  |
| 2.1      | Notation . . . . .                                                 | 9         |
| 2.2      | FIPS 205 SLH-DSA review . . . . .                                  | 9         |
| 2.3      | Per-validator standalone signature syntax . . . . .                | 10        |
| 2.4      | Threshold signature syntax (appendix forms) . . . . .              | 11        |
| 2.5      | Shamir secret sharing over GF(257) (appendix forms) . . . . .      | 11        |
| 2.6      | Security games . . . . .                                           | 12        |
| <b>3</b> | <b>Construction</b>                                                | <b>13</b> |
| 3.1      | Hash-domain separation . . . . .                                   | 13        |
| 3.2      | Per-validator keypair generation . . . . .                         | 13        |
| 3.3      | Per-validator signing primitive . . . . .                          | 14        |
| 3.4      | Aggregate certificate construction . . . . .                       | 14        |
| 3.5      | Aggregate certificate verification . . . . .                       | 15        |
| 3.6      | Canonical wire codec (MAGS / MAGG) . . . . .                       | 15        |
| 3.7      | Verify (FIPS 205 passthrough) . . . . .                            | 16        |
| 3.8      | Constant-time discipline . . . . .                                 | 16        |
| 3.9      | Identifiable abort (consensus layer) . . . . .                     | 17        |
| 3.10     | Wire sizes . . . . .                                               | 17        |
| <b>4</b> | <b>Security argument</b>                                           | <b>18</b> |
| 4.1      | Byte-identity to single-party FIPS 205 . . . . .                   | 18        |
| 4.2      | Multi-validator EUF-CMA . . . . .                                  | 19        |
| 4.3      | Trust posture . . . . .                                            | 19        |
| 4.4      | Constant-time posture . . . . .                                    | 20        |
| 4.5      | Composition with Polaris . . . . .                                 | 20        |
| 4.6      | Proof roadmap . . . . .                                            | 20        |
| <b>5</b> | <b>Implementation</b>                                              | <b>21</b> |
| 5.1      | File layout (standalone primary) . . . . .                         | 22        |
| 5.2      | Dependencies . . . . .                                             | 23        |
| 5.3      | Build and test discipline . . . . .                                | 23        |
| 5.4      | Headline regression: TestMagnetar_Wire_FIPS205Verifiable . . . . . | 24        |
| 5.5      | Configuration-as-data . . . . .                                    | 24        |
| 5.6      | Concurrency posture . . . . .                                      | 24        |
| 5.7      | Lux Polaris integration . . . . .                                  | 25        |
| <b>6</b> | <b>Evaluation</b>                                                  | <b>25</b> |
| 6.1      | Parameter sets . . . . .                                           | 25        |
| 6.2      | KAT determinism . . . . .                                          | 26        |
| 6.3      | Headline test: TestMagnetar_Wire_FIPS205Verifiable . . . . .       | 26        |
| 6.4      | Per-signature latency . . . . .                                    | 26        |
| 6.5      | Wire bandwidth . . . . .                                           | 27        |
| 6.6      | Empirical claims verified . . . . .                                | 27        |
| <b>7</b> | <b>Comparison to Pulsar and Corona</b>                             | <b>28</b> |
| 7.1      | Different threshold mechanics per primitive . . . . .              | 28        |
| 7.2      | Mathematical independence . . . . .                                | 28        |
| 7.3      | Performance comparison . . . . .                                   | 28        |
| 7.4      | Lifecycle composition . . . . .                                    | 29        |

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| 7.5       | Trust posture: what differs across lanes . . . . .                  | 29        |
| 7.6       | Where each lane is the right answer . . . . .                       | 30        |
| <b>8</b>  | <b>Path to public-DKG threshold HBS (open research)</b>             | <b>30</b> |
| 8.1       | Why FROST-style threshold does not apply . . . . .                  | 30        |
| 8.2       | Candidate sub-protocols for public-DKG THBS . . . . .               | 31        |
| 8.3       | Suggested public-DKG THBS direction . . . . .                       | 32        |
| 8.4       | What public-DKG THBS would and would not solve . . . . .            | 32        |
| 8.5       | Adjacent literature . . . . .                                       | 33        |
| <b>9</b>  | <b>Related work</b>                                                 | <b>33</b> |
| 9.1       | Threshold hash-based signatures . . . . .                           | 33        |
| 9.2       | Per-validator aggregate signatures for PQ consensus . . . . .       | 34        |
| 9.3       | Lattice-family threshold signatures . . . . .                       | 34        |
| 9.4       | Schnorr-family threshold (FROST) . . . . .                          | 35        |
| 9.5       | Threshold ECDSA (CGGMP21) . . . . .                                 | 36        |
| 9.6       | BLS aggregate . . . . .                                             | 36        |
| 9.7       | Standards landscape . . . . .                                       | 36        |
| 9.8       | Comparison table . . . . .                                          | 36        |
| 9.9       | Where Magnetar’s contribution lives . . . . .                       | 37        |
| <b>10</b> | <b>Conclusion</b>                                                   | <b>38</b> |
| 10.1      | What Magnetar v0.5.2 is . . . . .                                   | 38        |
| 10.2      | What Magnetar v0.5.2 is not . . . . .                               | 38        |
| 10.3      | Path to full Tier A . . . . .                                       | 39        |
| 10.4      | Closing . . . . .                                                   | 39        |
| <b>A</b>  | <b>THBS — TEE-Only Threshold HBS Construction</b>                   | <b>40</b> |
| A.1       | THBS protocol surface . . . . .                                     | 40        |
| A.2       | Construction-equivalence to single-party FIPS 205 . . . . .         | 41        |
| A.3       | Where THBS lives in the Lux stack . . . . .                         | 41        |
| <b>B</b>  | <b>Reveal-and-Aggregate over the SLH-DSA Scheme Seed (TEE-only)</b> | <b>42</b> |
| B.1       | Reveal-and-aggregate protocol surface . . . . .                     | 42        |
| B.2       | Byte-equality to single-party FIPS 205 . . . . .                    | 43        |
| B.3       | Where reveal-and-aggregate lives in the Lux stack . . . . .         | 43        |
| B.4       | Operational hardening (TEE deployment) . . . . .                    | 44        |

# 1 Introduction

Post-quantum threshold signatures are the load-bearing primitive of any blockchain consensus design that wants to survive a cryptographically relevant quantum adversary. The classical workhorses — BLS aggregate [15, 12] and ECDSA with CGGMP21 [17] — fail catastrophically under Shor’s algorithm, and the threshold replacements available for the post-quantum era do not all ground their security in the same underlying mathematical assumption. Defense in depth at the cryptographic primitive layer requires not just “a PQ scheme,” but a portfolio of PQ schemes whose security assumptions are *algorithmically independent*: a break of any one family does not propagate to the others.

The Lux Polaris cert profile [35] encodes this discipline in production. Polaris is the maximum-assurance Quasar finality profile that pairs three independent threshold lanes:

- **Pulsar** — threshold Module-LWE [28, 14], the lattice lane in  $\mathbb{Z}_q[X]/(X^{256} + 1)$  with a 48-bit prime;
- **Corona** — threshold Ring-LWE [30, 14], a structurally distinct lattice instantiation that does not share Pulsar’s module parameter choice; and
- **Magnetar** (this paper) — per-validator standalone FIPS 205 SLH-DSA [44], the hash-based lane whose security rests on SHA-3 / Keccak collision resistance and preimage resistance rather than on any structured lattice assumption.

The three legs are deliberately drawn from *different mathematical families*: a polynomial-time algorithm against module-lattice problems would break Pulsar and Corona simultaneously but leave Magnetar intact; a polynomial-time preimage attack on Keccak would break Magnetar but leave the lattice lanes intact; a single quantum oracle that defeats both families is, on current evidence, an inconsistent ask. Polaris finality requires *all three* lanes to verify, so an adversary must break the conjunction — a strictly harder task than breaking any single primitive.

Magnetar is the third leg. This paper specifies the construction at [github.com/luxfi/magnetar](https://github.com/luxfi/magnetar) v0.5.2 (per-validator standalone public-BFT primary primitive; canonical MAGS/MAGG wire codec; stateless `VerifyBytes` dispatcher), the byte-identity argument against single-party FIPS 205, and the Polaris-composition discipline that binds Magnetar to Pulsar and Corona under a common transcript.

## 1.1 What Magnetar is

Magnetar’s primary cryptographic primitive is *per-validator standalone FIPS 205 SLH-DSA* [44]. SLH-DSA is the NIST hash-based signature standard (formerly SPHINCS+ [8]); it is stateless, requires no trapdoor or structured ring, and inherits its security from the standard properties of cryptographic hash functions (collision resistance, preimage resistance, second-preimage resistance, and the Boneh–Shoup [13] sub-exponential preimage bound under quantum adversaries).

The Magnetar public-BFT construction departs deliberately from the threshold templates of FROST [33, 20], Pulsar [28], or Corona [30]. SLH-DSA has *no linear share-aggregation identity*: there is no algebraic operation analogous to FROST’s  $z = \sum_i z_i$  that produces a valid SLH-DSA signature from per-party contributions on the same message. The literature is uniform on this point. Cozzo-Smart (EUROCRYPT 2019, “Sharing the LUOV”) establish that every internal SHAKE/SHA-2 evaluation in a hash-based signature is a non-linear function of the seed, so any genuine  $t$ -of- $n$  scheme producing a single FIPS 205-shaped signature requires either (i) reconstructing the seed inside a trusted aggregator (the reveal-and-aggregate trust model, which puts the aggregator in the TCB), or (ii) running a full MPC over the entire SHA-256/SHAKE hash tree (open research, with cost proportional to the  $\sim 10^4$  hash invocations per SLH-DSA-SHAKE-192s signature). Bonte-Smart-Tan (2023, “Threshold SPHINCS+”) confirm the infeasibility analysis. NIST IR 8214 [45] classifies SLH-DSA as “highest threshold-MPC cost” among FIPS 203/204/205.

The architecturally correct primitive for *public-BFT consensus* under these constraints is therefore  $N$  standalone signatures collected at the consensus layer. Each validator  $i$  runs FIPS 205 KEYGEN once, persists  $(sk_i, pk_i)$ , and contributes a single-party  $\text{SIGN}_{\text{det}}(sk_i, \mu, ctx)$  to each consensus message. The consensus layer collects the  $N$  signatures into a `ValidatorAggregateCert` carrying the parallel triples  $(Signers, PubKeys, Sigs)$  and a known validator registry binding each  $Signers[i]$  to its registered  $pk_i$ . The quorum decision (“count  $\geq$  threshold”) lives at the consumer, not in the primitive; the primitive reports the validated count.

**Byte-identity to single-party FIPS 205.** Each  $\sigma_i$  emitted by `magnetar.ValidatorSign` (with `rng=nil`) is byte-identical to what a centralized FIPS 205 `SIGNDETERMINISTIC` would emit on the same  $(sk_i, \mu, ctx)$ . This is the load-bearing operational property: every SLH-DSA verifier in the wild (`cloudflare/circl/sign/slhdsa` [19], the NIST FIPS 205 reference, `openssl-pq` builds, `libsignal-pq`) accepts a Magnetar signature byte-for-byte after the consensus wire envelope is stripped. The byte-identity is empirically pinned by `TESTMAGNETAR_WIRE_FIPS205VERIFY` at `ref/go/pkg/magnetar/wire_test.go:166–220`: the test generates a keypair via `PerValidatorKeypair`, signs via `ValidatorSign`, frames both via the MAGS / MAGG wire codec, strips the 11-byte canonical header to recover the raw FIPS 205 payload bytes, and feeds them to `cloudflare/circl/sign/slhdsa` with no Magnetar code path on the verifier side. The test passes for SHAKE-192s, SHAKE-192f, and SHAKE-256s under `-count=1` and `-race`.

**Public-BFT contract.** The per-validator standalone primitive carries the property the cryptographic-diversity argument of §1.2 requires:

1. No DKG, no dealer, no shared seed. Each validator generates its own keypair on its own host with its own RNG. Losing  $sk_i$  is a local failure — the validator drops out of the quorum until rotating to a fresh keypair — not a threshold-secret reconstruction event.
2. No aggregator-in-TCB. The consensus layer never holds anyone’s secret. `ValidatorBatchVerify` and `VerifyAggregateCert` are pure public-input functions.
3. No new verification code. Relying parties run any unmodified FIPS 205 verifier against the wire payload; the Magnetar envelope is opaque to them.
4. Wire shape that maps directly to Z-Chain Groth16 rollup witness layout. The `ValidatorAggregateCert` parallel-slices form  $(Mode, Signers, PubKeys, Sigs)$  is the wire shape the rollup prover consumes verbatim.

The wire-size cost is  $N \times |\sigma|$  before the rollup. For SHAKE-192s ( $|\sigma| = 16,224$  B) at a  $K = 21$  Polaris quorum that is  $\sim 332$  KiB. The Z-Chain Groth16 rollup compresses this to  $\sim 192$  B before the certificate hits long-term storage. The rollup is a separate primitive specified at the lux-zchain layer; Magnetar emits the rollup-friendly parallel-slices shape but does not own the rollup.

## 1.2 Why hash-based

Lattice cryptography (Module-LWE, Ring-LWE, NTRU) is the consensus post-quantum direction for performance reasons — ML-KEM [42], ML-DSA [43], and FALCON [25] all operate in milliseconds, ship signatures in the few-KB range, and parallelise well. But the lattice family rests on a single set of related-problem assumptions (Module-LWE, Ring-LWE, NTRU, Module-SIS) whose hardness is conjectured but whose ultimate security margin is the subject of ongoing cryptanalytic refinement. The cryptanalytic surface includes BKZ block-size cost models [6, 34], dual sieves [1], structured-lattice algebraic attacks on cyclotomic rings, and quantum-amplified sieves whose constants remain in modest motion. A polynomial-time algorithm for any of these problems would invalidate the entire lattice signature ecosystem simultaneously.

Hash-based signatures rest on assumptions that are *older, more broadly studied, and structurally orthogonal*. FIPS 205 SLH-DSA’s security reduces to:

- collision resistance of the hash function family (SHA-3 / SHAKE for the FIPS 205 SHAKE modes Magnetar targets);
- preimage resistance of the same;
- the absence of unforeseen structural weaknesses in the WOTS+ and FORS one-time / few-time signature schemes that build on the hash function.

These properties have been studied for decades. SHA-3 in particular emerged from a multi-year open competition [9]; its internal structure (Keccak’s sponge construction over a permutation) is qualitatively distinct from Merkle–Damgård, BLAKE, and the keyed-permutation Romulus and Ascon families. A quantum attack that defeats SHA-3 is one that defeats hash functions *as a class*, a strictly stronger ask than defeating any one family.

The trade-off is performance: SLH-DSA signatures are 16–36 KB, signing takes hundreds of milliseconds to seconds on commodity hardware, and the construction is the slowest of the FIPS-standardised PQ schemes. For a *single* hot-path signature this would be a non-starter. For the *Polaris third lane* it is precisely the trade-off we want: Polaris’s BLS lane sets the latency floor, Pulsar sets the per-block PQ floor, and Magnetar shoulders the diversity insurance — it does not need to be the fastest, only to be *independent* of the other two.

### 1.3 What is novel

The cryptographic primitives in this paper are not new. FIPS 205 [44] is the NIST-standardised SLH-DSA. A per-validator standalone signature is just a single-party FIPS 205 signature, and  $N$ -of- $N$  collection at a consensus layer is the canonical pattern for ML-DSA (already in production via the `QuasarCert.MLDSAProof` field) and BLS aggregate.

What `luxfi/magnetar` contributes is the *composition*:

1. An explicit-semantic public-BFT API surface at `ref/go/pkg/magnetar/standalone.go` — `PerValidatorKeypair`, `ValidatorSign`, `ValidatorBatchVerify`, `ValidatorAggregateCert`, `BuildAggregateCert`, `VerifyAggregateCert` — that decomplexes the public-BFT signing primitive from the TEE-only threshold forms below. One signing primitive  $\rightarrow$  one keypair  $\rightarrow$  no shared state  $\rightarrow$  no aggregator-in-TCB. The quorum decision lives at the consumer.
2. A canonical wire codec (MAGS for signatures, MAGG for group keys; 11-byte fixed header carrying magic, version, mode, and big-endian payload length) at `ref/go/pkg/magnetar/wire.go`, with a stateless dispatcher `VerifyBytes(groupKey, msg, sig)` that accepts wire bytes from the threshold daemon and verifies under FIPS 205 with no out-of-band state. The codec is siblinged with Pulsar’s PULS/PULG and Corona’s CORS/CORG canonical wire framings, so the consensus-layer dispatcher is uniform across the three Polaris lanes.
3. A byte-identity gate `TESTMAGNETAR_WIRE_FIPS205VERIFIABLE` that pins per-validator  $\sigma_i$  to single-party `cloudflare/circl/sign/slhdsa.Verify` across all three FIPS 205 SHAKE parameter sets. The verifier-side code path is the unmodified `circl` reference; no Magnetar code runs there. This is the empirical Class-N1-analog property translated to the standalone construction.
4. Hash-function domain separation across every Magnetar cSHAKE256 invocation — cSHAKE256 with function name “Magnetar” and per-purpose customisation strings — so cross-context replay attacks reduce to cSHAKE256 collision resistance.
5. A Polaris cert-profile composition: Magnetar joins Pulsar and Corona as the third lane of the maximum-assurance profile, sharing the LSS lifecycle adapter contract [50] and the Prism transcript-binding discipline [28]. The `ValidatorAggregateCert` parallel-slices shape is designed to feed the Z-Chain Groth16 rollup that compresses  $N \times |\sigma|$  to a constant-size certificate.
6. Honest demotion of the threshold forms. Reveal-and-aggregate seed reconstruction and the THBS construction over selected FORS / WOTS+ elements both ship in the same artefact under `ref/go/pkg/thbs/` and `combine.go`, but they are explicitly TEE-required by policy. The deployment runbook directs public-BFT consumers to `ValidatorSign` as

the only public-BFT-safe entry point; the threshold forms are appropriate for M-Chain bridge custody and A-Chain confidential compute, where a TEE-attested host is in the TCB by policy. See Appendix A and Appendix B.

## 1.4 Trust model summary

The per-validator standalone primitive has a clean trust model: it is the trust model of single-party FIPS 205 SLH-DSA, replicated independently  $N$  times. We state it explicitly.

**In the TCB at sign time:** validator  $i$ 's own host. Each validator holds its own  $sk_i$  locally; losing  $sk_i$  to an attacker allows that attacker to sign as validator  $i$  until  $pk_i$  is rotated out of the validator-set registry. This is the same exposure as a classical secp256k1 validator key, and the same operational mitigations apply (HSM-backed key storage, key rotation, slashing evidence for double-signing).

**NOT in the TCB:** the consensus layer, the `ValidatorAggregateCert` aggregator path, the `VerifyAggregateCert` verifier, the wire codec, or any other validator. The aggregation and verification primitives are pure public-input functions over  $(PubKeys, Sigs, Signers, \mu, registry)$ . There is no shared secret to compromise.

**Cross-validator independence.** Because each validator holds an independent keypair, compromising  $sk_i$  for validator  $i$  gives the attacker exactly one quorum vote. Forging a quorum requires compromising  $\geq threshold$  distinct validator hosts, which is the same Byzantine-fault threshold the rest of the consensus stack is sized to. No threshold-secret reconstruction event exists to be raced against; an attacker who steals  $threshold - 1$  keypairs cannot recover or forge with the remaining honest keypairs.

**Quantum threat.** The per-validator standalone primitive inherits FIPS 205's post-quantum security budget:  $\sim 2^{192}$  classical /  $\sim 2^{128}$  quantum at SHAKE-192s, and  $\sim 2^{256}$  classical /  $\sim 2^{128}$  quantum at SHAKE-256s. A quantum adversary that can mount a  $2^{128}$ -Grover attack against a single validator's SHA-3 evaluations would need  $\sim 2^{128}$  quantum operations per validator to forge that validator's signatures, and would still control only one quorum vote per forgery target.

**Threshold forms (TEE-only).** The threshold forms shipped in the same artefact — THBS v1 (Appendix A) and reveal-and-aggregate over the SLH-DSA scheme seed (Appendix B) — carry their own trust models in which a dealer or aggregator holds master secret material in memory. These forms are explicitly NOT public-BFT-safe and require hardware-attested confidential compute (AMD SEV-SNP, Intel TDX, or equivalent). They are documented in the appendices as companion primitives for M-Chain bridge custody and A-Chain confidential compute.

## 1.5 What this paper specifies

Section 2 reviews FIPS 205 SLH-DSA, the threshold-signature syntax used by the appendices, and the security games. Section 3 gives the per-validator standalone construction byte-exact in pseudocode, keyed to the Go canonical at `ref/go/pkg/magnetar/standalone.go` and `wire.go` byte-for-byte. Section 4 states the security argument: byte-identity to single-party FIPS 205, the  $N$ -of- $N$  verification semantics, and the existential unforgeability inheritance from FIPS 205 UF-CMA per validator. Section 5 surveys the reference implementation. Section 6 reports



parameter sets and per-signature latency from the production test suite. Section 7 locates Magnetar alongside Pulsar and Corona in the Polaris cert profile. Section 8 surveys the open research path to a true public-DKG threshold HBS that would not require TEE. Section 9 surveys the (sparse) threshold-SLH-DSA literature and the broader threshold-hash-based-signature work. Section 10 closes.

Appendix A documents the THBS (true threshold hash-based signature) construction per McGrew et al. ePrint 2019/793 shipped at `ref/go/pkg/thbs/`, with explicit TEE-only-by-policy framing. Appendix B documents the reveal-and-aggregate construction over the SLH-DSA scheme seed shipped at `ref/go/pkg/magnetar/combine.go`, with the same TEE-only-by-policy framing.

**Artefact status.** At the time of writing, the artefact is `luxfi/magnetar v0.5.2` (commit `1c84519`; tagged 2026-05-31). The per-validator standalone primary primitive shipped at `v0.5.0`; the canonical MAGS/MAGG wire codec and the stateless `VerifyBytes` dispatcher shipped at `v0.5.1`; `v0.5.2` added the integration entry points for the threshold daemon and pinned the byte-identity regression test across all three parameter sets under `-count=1` and `-race`. The Tier-A documentation shape (12-document package mirroring Pulsar’s structure) is in-tree. Mechanised proof artefacts (EasyCrypt theories for FIPS 205 UF-CMA inheritance, Lean bridges for the wire-codec byte-identity lemma, Jasmin sources for the constant-time hot paths) and external cryptographer audit are the remaining gates to full Tier A.

## 2 Preliminaries

### 2.1 Notation

- $\lambda$  — security parameter (set to 192 for SHAKE-192s, 256 for SHAKE-256s).
- $N$  — validator-set size at the consensus layer (the primary primitive of this paper is  $N$ -of- $N$  collection; a quorum policy of size  $K \leq N$  filters the count at the consumer).
- $P_i$  — validator  $i$ ,  $1 \leq i \leq N$ . Each validator is identified by a 32-byte `NodeID`.
- $(sk_i, pk_i)$  — validator  $i$ ’s FIPS 205 SLH-DSA keypair. Each validator’s keypair is independent of every other validator’s; there is no shared seed, no DKG, and no dealer.
- $\sigma_i$  — validator  $i$ ’s FIPS 205 SLH-DSA signature (the wire bytes) on the consensus message.
- $\mu$  — the message being signed;  $ctx$  — the FIPS 205 context string (0–255 bytes).
- $registry : \text{NodeID} \rightarrow \{0, 1\}^{|pk|}$  — the consensus layer’s authoritative validator-set registry, binding `NodeIDi` to  $pk_i$ .
- cSHAKE256 — the cSHAKE256 customisable hash from NIST SP 800-185 [31], with function-name string “Magnetar” and per-context customisation tags.
- $\parallel$  — byte concatenation;  $\oplus$  — bitwise XOR.

**Appendix notation.** The appendices on threshold forms introduce additional symbols:  $n$  for committee size,  $t$  for reconstruction threshold,  $\text{GF}(p)$  for the Shamir field with  $p = 257$ ,  $S$  for the shared SLH-DSA scheme seed, and  $share_i$  for party  $i$ ’s share. Those symbols are local to the appendices and do not appear in the main per-validator standalone construction.

### 2.2 FIPS 205 SLH-DSA review

FIPS 205 [44] (formerly SPHINCS+ [8]) is the NIST stateless hash-based signature standard. The Magnetar reference implementation targets the FIPS 205 SHAKE parameter sets:

Table 1: FIPS 205 SHAKE parameter sets exposed by Magnetar v0.5. Values from FIPS 205 [44] §10.1 Table 2.

| Parameter set      | NIST PQ Cat. | $n$ (WOTS+) | $h$ | $d$ | $ pk $ | $ \sigma $ |
|--------------------|--------------|-------------|-----|-----|--------|------------|
| SLH-DSA-SHAKE-192s | 3            | 24 B        | 63  | 7   | 48 B   | 16 224 B   |
| SLH-DSA-SHAKE-192f | 3            | 24 B        | 66  | 22  | 48 B   | 35 664 B   |
| SLH-DSA-SHAKE-256s | 5            | 32 B        | 64  | 8   | 64 B   | 29 792 B   |

The FIPS 205 construction is built from the following layered components, top to bottom:

- **XMSS-HT (hypertree)**: a tree of  $d$  XMSS layers. The leaves of the root layer are XMSS public keys; the leaves of the bottom layer are FORS public keys. Every layer has height  $h/d$ .
- **XMSS (eXtended Merkle Signature Scheme)**: a binary Merkle tree where each leaf is a WOTS+ public key.
- **WOTS+ (Winternitz One-Time Signature)**: a one-time signature whose security reduces to hash function preimage and second-preimage resistance.
- **FORS (Forest of Random Subsets)**: the few-time signature scheme used at the bottom of the hypertree.
- **Tweakable hash  $F, H, T_\ell$** : the cryptographic primitives, instantiated as SHAKE256 for the SHAKE-family Magnetar targets.

**Key generation.**  $\text{KEYGEN}$  [44] §9.1 samples a  $3n$ -byte secret triple  $(SK.seed, SK.prf, PK.seed)$  and computes  $PK.root = T_h(SK.seed, PK.seed, \text{ADRS})$  as the XMSS-HT root. The public key is  $pk = (PK.seed, PK.root)$ ; the secret key is  $sk = (SK.seed, SK.prf, PK.seed, PK.root)$ . The total scheme-seed size (the input to  $\text{DERIVEKEY}$ ) is  $4n$  bytes: 96 for  $n = 24$  (192s/192f) and 128 for  $n = 32$  (256s).

**Deterministic signing.**  $\text{SIGN}_{\text{det}}(sk, \mu, ctx)$  [44] §9.2 computes a randomiser  $R \leftarrow \text{PRF}_{SK.prf}(ctx \parallel \mu)$  and proceeds deterministically. The output is byte-identical across calls with the same  $(sk, \mu, ctx)$ . The Magnetar per-validator standalone primitive uses this variant; the byte-identity claim against `cloudflare/circl/sign/slhdsa.Verify` pins to this determinism.

**Randomised signing.**  $\text{SIGN}_{\text{rand}}(sk, rng, \mu, ctx)$  [44] §9.2 samples a fresh  $n$ -byte randomiser from  $rng$ . The Magnetar `ValidatorSign` entry point routes to the deterministic variant by default (`rng=nil`); passing a non-nil reader selects the randomised variant for callers who specifically want hedging against fault injection on the  $PRF$  derivation.

**Verification.**  $\text{VERIFY}(pk, \mu, \sigma, ctx)$  [44] §9.3 walks the XMSS-HT root, recomputes  $PK.root$  from the FORS and XMSS authentication paths in  $\sigma$ , and accepts iff it matches the  $PK.root$  embedded in  $pk$ . The verifier consumes no secret material; *any unmodified FIPS 205 verifier accepts a Magnetar signature with no code change*.

### 2.3 Per-validator standalone signature syntax

The primary primitive of this paper is a per-validator standalone SLH-DSA construction. The syntax is the single-party FIPS 205 syntax, replicated independently  $N$  times, plus an aggregate-certificate shape at the consensus layer.

**Definition 1** (Per-validator standalone signature scheme).  $\Pi_{\text{Magnetar}}^{\text{std}}$  is a tuple of probabilistic polynomial-time algorithms  $(\text{KEYGEN}, \text{SIGN}, \text{BATCHVERIFY}, \text{BUILDCERT}, \text{VERIFYCERT})$ :

- $\text{KEYGEN}(1^\lambda)$ : each validator  $i$  runs this independently on its own host. Outputs an SLH-DSA keypair  $(sk_i, pk_i)$ . No interaction.
- $\text{SIGN}(sk_i, \mu, ctx)$ : validator  $i$ 's signing primitive. Returns  $\sigma_i = \text{SLHDSA.SIGNDETERMINISTIC}(sk_i, \mu, ctx)$ . Deterministic.
- $\text{BATCHVERIFY}(\{pk_i\}, \mu, \{\sigma_i\}) \rightarrow \text{bitmap}$ : returns a Boolean bitmap indicating per-signer FIPS 205 verifier acceptance. Pure function of public inputs.
- $\text{BUILDCERT}(\text{Mode}, \{\text{NodeID}_i, pk_i, \sigma_i\}_{i \in S})$ : assembles the on-wire `ValidatorAggregateCert` carrying the parallel-slices triple  $(\text{Signers}, \text{PubKeys}, \text{Sigs})$  for the contributing set  $S$ .
- $\text{VERIFYCERT}(\text{cert}, \mu, \text{registry}) \rightarrow \text{count}$ : enforces registry binding (drops unknown signers and pubkey mismatches), dedupes repeated signers, calls `BATCHVERIFY` on the verifiable subset, and returns the count of valid signers. The quorum decision “ $\text{count} \geq K$ ?” lives at the consumer.

The deliberate choice that `SIGN` is the unmodified FIPS 205 `SIGNDETERMINISTIC` — not a Magnetar-specific envelope — is the load-bearing operational property: any deployment shipping a Magnetar signer needs to add no new verification code on the consuming side. Every SLH-DSA verifier in the wild (`cloudflare/circl/sign/slhdsa` [19], the FIPS 205 reference, `openssl-pq`, `libsignal-pq`) accepts Magnetar signatures unmodified after the wire envelope is stripped.

## 2.4 Threshold signature syntax (appendix forms)

The threshold forms documented in the appendices use the standard threshold-signature syntax of Canetti [16] and Boneh–Shoup [13] as specialised to SLH-DSA.

**Definition 2** (Threshold SLH-DSA scheme (appendix forms)).  $\Pi_{\text{Magnetar}}^{\text{thr}}$  is a tuple  $(\text{SETUP}, \text{SIGNSHARE}, \text{COMB})$

- **SETUP**: produces the joint group public key  $pk$  and per-party secret shares. For Appendix A this is `thbs.DealerDKG` (dealer-backed  $v1$ ; PVSS skeleton for  $v2$  at `thbs/dkg2/`). For Appendix B this is the three-round Magnetar DKG over the SLH-DSA scheme seed.
- **SIGNSHARE**: per-party contribution. For THBS this is the selected-element share release; for reveal-and-aggregate this is the two-round commit-and-reveal of the local Shamir share.
- **COMBINE**: deterministic aggregator function. For THBS this Lagrange-reconstructs the selected WOTS+ chain heads and FORS leaves; for reveal-and-aggregate this Lagrange-reconstructs the scheme seed.
- **VERIFY**: the unmodified FIPS 205 verifier of [44] §9.3.

The threshold forms preserve the FIPS 205 output shape — `COMBINE` output is byte-identical to single-party `SIGNDETERMINISTIC` on the implicit  $(\text{seed}, \mu, ctx)$  tuple — so a relying party sees no difference between a standalone  $\sigma_i$  and a threshold  $\sigma$  once both are unwrapped from the consensus envelope.

## 2.5 Shamir secret sharing over $\text{GF}(257)$ (appendix forms)

We instantiate Shamir secret sharing [51] byte-wise over  $\text{GF}(p)$  with  $p = 257$  for the threshold forms in Appendix B and (with an additional `cSHAKE256` mix on the output) Appendix A. The choice of  $p = 257$  is dictated by two constraints:

1.  $p > 255$ , so every byte value  $0, \dots, 255$  is a distinct field element.
2.  $p < 2^9$ , so each share value fits in a 9-bit lane, packed as one `uint16` per byte position on the wire.

The trade-off:  $p = 257$  admits at most  $p - 1 = 256$  distinct non-zero evaluation points, capping the committee size at 256 (constant `MaxCommittee257` in `params.go`). For the Lux validator committees this is comfortably above the production thresholds (Quasar operates at  $K \leq 21$  [35]).

**Sharing.** To share a `SEED_SIZE`-byte secret  $\mathbf{s} \in [0, 256]^{\text{SEED\_SIZE}}$  across  $n$  parties at threshold  $t$ :

1. For each byte position  $b \in [0, \text{SEED\_SIZE})$ , sample  $t-1$  random coefficients  $a_{b,1}, \dots, a_{b,t-1} \in \text{GF}(257)$  and define  $f_b(x) = \mathbf{s}[b] + \sum_{j=1}^{t-1} a_{b,j} x^j \pmod{257}$ .
2. For each recipient  $P_i$  at evaluation point  $x_i = i$  (one-indexed), compute  $\text{share}_i[b] = f_b(x_i) \pmod{257}$  for all  $b$ .
3. Wire-form: pack each  $\text{share}_i$  as a sequence of `SEED_SIZE` big-endian `uint16` values ( $2 \cdot \text{SEED\_SIZE}$  bytes total).

**Reconstruction.** Given any  $t$  shares  $\{(x_i, \text{share}_i)\}_{i \in Q}$  with  $|Q| \geq t$ , the constant term of each  $f_b$  is recovered by Lagrange interpolation at  $x = 0$ :

$$\mathbf{s}[b] = \sum_{i \in Q} \lambda_i^Q(0) \cdot \text{share}_i[b] \pmod{257}, \quad \lambda_i^Q(0) = \prod_{j \in Q, j \neq i} \frac{-x_j}{x_i - x_j} \pmod{257}. \quad (1)$$

The Lagrange coefficients  $\lambda_i^Q(0)$  are computed once per quorum using Fermat’s little theorem for modular inversion ( $a^{-1} \equiv a^{p-2} \pmod{p}$ ); see `shamir.go` lines 148–164.

**Information-theoretic privacy.** For any  $|Q| < t$ , the joint distribution of  $\{\text{share}_i\}_{i \in Q}$  is uniformly distributed over  $[0, 257)^{|Q| \cdot \text{SEED\_SIZE}}$  independent of  $\mathbf{s}$ . This is the standard Shamir privacy property [51, 27]: every  $t-1$ -subset of shares reveals zero information about the secret, as a matter of information theory (not computational complexity). This is the load-bearing secrecy guarantee of the appendix threshold forms against a coalition of  $t-1$  or fewer committee members.

**The GF(257)/byte-domain mismatch.** The Lagrange reconstruction (1) produces a value in  $[0, 257)$ ; the SLH-DSA scheme seed is byte-valued in  $[0, 256)$ . The single value 256 has no byte representation. The appendix forms handle this *constructively*: the Shamir reconstruction is fed into a cSHAKE256 mix

$$S = \text{cSHAKE256}(\text{byteSum}_{BE} \parallel \text{committee\_root}, N = \text{“Magnetar”}, S = \text{“MAGNETAR-SEED-SHARE-V1”}, 8). \quad (2)$$

which absorbs the GF(257)-valued `byteSum` as a  $2 \cdot \text{SEED\_SIZE}$ -byte big-endian buffer and emits a `SEED_SIZE`-byte  $S$ . The cSHAKE256 mix domain-separates the seed from raw Shamir output and gives a constant-length seed regardless of the value distribution of `byteSum`.

## 2.6 Security games

The per-validator standalone primitive’s security game is the single-party SLH-DSA UF-CMA game replicated  $N$  times in parallel. We state the formal definition.

**Definition 3** (Multi-validator EUF-CMA).  $\text{Game}_{\Pi_{\text{Magnetar}}^{\text{std}}}^{\text{multi-EUF-CMA}}(\mathcal{A}, \lambda, N)$ :

1. **Setup.** The challenger runs `KEYGEN` independently  $N$  times to obtain  $(sk_i, pk_i)$  for  $i = 1, \dots, N$ . The adversary  $\mathcal{A}$  commits to a corrupt set  $F \subset [N]$  with  $|F| < K$  at the outset, where  $K$  is the quorum threshold; the challenger gives  $\mathcal{A} \{(pk_i)\}_{i=1}^N \cup \{sk_i : i \in F\}$ .
2. **Queries.**  $\mathcal{A}$  adaptively requests signatures from honest validators on messages  $\mu_1, \dots, \mu_{Q_S}$ . For each query  $(i, \mu)$  with  $i \notin F$ , the challenger returns  $\sigma_i = \text{SIGN}(sk_i, \mu, \text{ctx})$ .  $\mathcal{A}$  may invoke the random oracle  $\mathcal{O}_H$  up to  $Q_H$  times.
3. **Forgery.**  $\mathcal{A}$  outputs  $(\mu^*, \text{cert}^*)$  and wins iff  $\text{VERIFYCERT}(\text{cert}^*, \mu^*, \text{registry}) \geq K$  and there exist  $K - |F|$  signers  $i \notin F$  whose signatures in  $\text{cert}^*$  are valid forgeries on  $\mu^* \notin \{\mu_j : (i, \mu_j) \in \text{queries}\}$ .

The advantage is  $\text{Adv}_{\Pi_{\text{Magnetar}}^{\text{std}}, \mathcal{A}}^{\text{multi-EUF-CMA}}(\lambda, N) = \Pr[\mathcal{A} \text{ wins}]$ .

The multi-validator forgery condition reduces to single-party SLH-DSA UF-CMA via a standard hybrid argument: each honest validator's keypair is independent, so any forgery on validator  $i$ 's signature yields a single-party SLH-DSA UF-CMA forgery on  $(pk_i, sk_i)$ . The reduction has a factor  $N$  loss for the hybrid (the standard hybrid argument over independent keypairs); the SLH-DSA UF-CMA bound at the SHAKE-192s parameter set ( $\leq 2^{-192}$  classical,  $\leq 2^{-128}$  quantum) absorbs this factor comfortably for any  $N \leq 2^{60}$ .

The threshold-form security games used in the appendices are the standard threshold EUF-CMA game in the aggregator-honest model (reveal-and-aggregate) or the slot-bound McGrew threshold-HBS game (THBS); we defer their formal statement to the appendices.

### 3 Construction

This section specifies the per-validator standalone construction byte-for-byte. The pseudocode is keyed to the Go canonical at `ref/go/pkg/magnetar/standalone.go` and `ref/go/pkg/magnetar/wire.go` at commit 1c84519 (v0.5.2). The threshold forms (THBS, reveal-and-aggregate) are specified in Appendix A and Appendix B.

#### 3.1 Hash-domain separation

Every cryptographic hash invocation in Magnetar is cSHAKE256 [31] with function-name string "Magnetar" and a per-purpose customisation string. The per-validator standalone path uses only the wire-codec canonical magic constants; the threshold forms use additional per-purpose tags listed in Table 2.

Table 2: Magnetar v0.5 cSHAKE256 domain separation. Every secret-bearing hash invocation is bound to exactly one tag; cross-tag replay reduces to cSHAKE256 collision resistance. Tags marked (*thresh*) appear only in the threshold-form appendices.

| Tag                        | Purpose                                                                 |
|----------------------------|-------------------------------------------------------------------------|
| MAGS (magic 0x4D414753)    | MAGS wire envelope for FIPS 205 signature payload (standalone primary)  |
| MAGG (magic 0x4D414747)    | MAGG wire envelope for FIPS 205 public-key payload (standalone primary) |
| MAGNETAR-DKG-COMMIT-V1     | committee-root digest (sorted committee binding) ( <i>thresh</i> )      |
| MAGNETAR-DKG-TRANSCRIPT-V1 | DKG transcript digest (Round-2 equivocation check) ( <i>thresh</i> )    |
| MAGNETAR-SIGN-R1-V1        | threshold-sign Round-1 commit $D_i$ ( <i>thresh</i> )                   |
| MAGNETAR-SIGN-MASK-V1      | per-attempt mask $r_i$ derivation ( <i>thresh</i> )                     |
| MAGNETAR-SEED-SHARE-V1     | Shamir coefficient stream + final seed mix ( <i>thresh</i> )            |

The protocol-wide domain separation context is the byte string "lux-magnetar-v0.5" (`tagDomainSep`). It is bound into the wire-codec magic constants and into every threshold-form transcript binding so that a separate Magnetar version (or a different Lux PQ primitive that happened to reuse the same hash function) cannot collide.

#### 3.2 Per-validator keypair generation

Each validator  $i$  runs `PerValidatorKeypair` once at validator-set onboarding, persists  $(sk_i, pk_i)$  to local HSM-backed storage, and registers  $pk_i$  in the consensus layer's validator-set commitment.

**Algorithm** (`standalone.go:197–199`, `keygen.go`).

1. Pull `params` for the target Mode (SHAKE-192s, SHAKE-192f, or SHAKE-256s).
2. Source  $4n$  bytes of entropy from `rng` (where  $n$  is the FIPS 205 “ $n$ ” parameter: 24 for 192, 32 for 256). If `rng=nil`, source from `crypto/rand`.
3. Pass the entropy buffer to `circl.slhdsa.Scheme(params.ID).DeriveKey`.
4. Output  $(sk_i, pk_i)$  in the FIPS 205 wire form:  $sk_i = (SK.seed, SK.prf, PK.seed, PK.root)$  of length  $4n$ ;  $pk_i = (PK.seed, PK.root)$  of length  $2n$ .

**Determinism.** If `rng` is a deterministic reader (e.g., a `detReader` seeded by the canonical test fixture), the output keypair is byte-identical across calls. This is the property KAT regeneration relies on.

**Independence.** No shared state. No correlation between  $(sk_i, pk_i)$  and  $(sk_j, pk_j)$  for  $i \neq j$ . Each keypair is generated on its own host with its own RNG.

### 3.3 Per-validator signing primitive

`ValidatorSign` is the canonical signing primitive for the public-BFT path. It is a thin wrapper over the FIPS 205 reference, exposed with a public-BFT-explicit signature.

**Algorithm** (`standalone.go:231–244`, `sign.go`).

1. Verify  $sk_i$  is non-nil and the embedded mode matches an installed `Params`.
2. Pull the `slhdsa.Scheme` for the embedded mode.
3. If `rng=nil`, invoke `Scheme.SignDeterministic(sk_i, msg, ctx=nil)`. Otherwise invoke `Scheme.SignRandomized(sk_i, rng, msg, ctx=nil)`.
4. Return the raw FIPS 205 signature bytes (`params.SignatureSize`: 16,224, 35,664, or 29,792 depending on mode).

**Context binding.** The `ctx` parameter is intentionally omitted at the `ValidatorSign` surface: the consensus layer binds chain-id / block-height / epoch into the message  $\mu$  upstream (the QuasarCert transcript-hash pattern). The `Sign` entry point in `sign.go` retains the `ctx` parameter for callers that need explicit context binding; the `ValidatorSign` wrapper passes `ctx=nil`.

**Determinism.** With `rng=nil`,  $\sigma_i$  is byte- identical to what a single-party FIPS 205 signer would emit on  $(sk_i, \mu, ctx = \varepsilon)$ . This is the load-bearing property pinned by `TESTMAGNETAR_WIRE_FIPS205` at `wire_test.go:166–220` (cf. §4.1).

### 3.4 Aggregate certificate construction

The consensus layer collects validator signatures into a `ValidatorAggregateCert` for transport over the wire. The cert is intentionally a parallel-slices shape (not a nested per- signer record list) because it maps directly to the Z-Chain Groth16 rollout witness layout.

**Wire layout** (`standalone.go:415–432`, `wire.go`).

$$\begin{aligned}
 \text{ValidatorAggregateCert} = & \text{Mode}_1 \parallel N_{BE,4} \parallel \text{Signers}[0]_{32} \parallel \cdots \parallel \text{Signers}[N-1]_{32} \\
 & \parallel \text{PubKeys}[0]_{|pk|} \parallel \cdots \parallel \text{PubKeys}[N-1]_{|pk|} \\
 & \parallel \text{Sigs}[0]_{|\sigma|} \parallel \cdots \parallel \text{Sigs}[N-1]_{|\sigma|}.
 \end{aligned}$$

**Algorithm BuildAggregateCert (standalone.go:443-478).**

1. Validate `params` matches one of the canonical parameter tables.
2. Validate the three parallel slices (*signers*, *pubKeys*, *sigs*) have the same length  $N \geq 1$ .
3. For each  $i \in [0, N)$ , validate  $|pubKeys[i]| = params.PublicKeySize$  and  $|sigs[i]| = params.SignatureSize$ .
4. Defensively copy each *pubKeys*[*i*] and *sigs*[*i*] (the caller's slices may be aliased; the cert is wire-stable and must not move under the caller).
5. Return the `ValidatorAggregateCert` record.

**Independent verifiability.** Each  $\sigma_i$  in the cert verifies independently against  $pk_i$ ; there is no cross-signer algebraic coupling. A relying party can verify any subset of the cert (e.g., for incremental quorum-detection) without sharing any secret state with other relying parties.

### 3.5 Aggregate certificate verification

`VerifyAggregateCert` is the consumer-facing verification primitive. It enforces registry binding (dropping unknown signers and pubkey mismatches), dedupes repeated signers, runs the parallel batch verify, and returns the count of valid signers. The quorum decision lives at the consumer.

**Algorithm (standalone.go:519-613).**

1. Validate `cert` is non-nil and the embedded `Mode` matches a canonical parameter set.
2. Validate  $N \geq 1$  and the parallel slices have matching lengths.
3. For each  $i \in [0, N)$ , validate  $|cert.PubKeys[i]| = params.PublicKeySize$  and  $|cert.Sigs[i]| = params.SignatureSize$ .
4. *Phase 1 (entry classification).* For each  $i$ , dedupe against `seen`, look up `registered = knownValidators[cert.Signers[i]]`; mark the entry verifiable only if (a) the lookup succeeds, (b)  $|registered| = params.PublicKeySize$ , and (c) `ctEqualBytes(cert.PubKeys[i], registered)`. Both cases (a)/(b)/(c) failing are counted as `INVALID`, not fatal.
5. *Phase 2 (parallel batch verify).* Collect the verifiable entries into flat slices and invoke `ValidatorBatchVerify`, which dispatches serial or goroutine-parallel based on the `verifyAggregatedConc` constant (currently  $\sim 16$ ).
6. *Phase 3 (tally).* Count verifiable + valid entries. Return the count.

**Defense-in-depth choices.**

1. **Unknown signers counted as `INVALID`, not fatal.** A cert carrying a signature from a stale or unknown validator does not crash the verifier; the verifier reports the count, and the quorum policy at the consumer decides whether to accept. This is the right discipline when certs may be Z-Chain Groth16-rolled-up over a stale validator registry: the verifier must not abort the entire batch on one stale entry.
2. **Pubkey mismatches counted as `INVALID`, not fatal.** Same rationale. An attacker who slips a `SignedBundle` from an unknown validator into the cert does not gain a count but does not crash the verifier.
3. **Duplicate signers deduped first-wins.** The first occurrence wins; subsequent occurrences are silently dropped (not counted twice, not counted as invalid). The consensus layer can collect proof of double-signing separately via per-signer `Sigs` comparison.
4. **Constant-time pubkey comparison.** `ctEqualBytes` short-circuits at the wire-length validation and otherwise runs a constant-time byte-by-byte XOR-then-OR reduction. No secret-dependent timing.

### 3.6 Canonical wire codec (MAGS / MAGG)

The wire codec at `wire.go` provides byte-stable framing for individual signatures (MAGS) and group keys (MAGG), with a stateless verifier dispatcher (`VerifyBytes`) for consensus-layer



consumption.

### MAGS signature envelope.

$$\begin{aligned} \text{MAGS\_frame} = & 0x4D4147 \parallel 0x53 \parallel \text{version}_{BE,2} \parallel \text{mode}_1 \parallel \text{payloadLen}_{BE,4} \\ & \parallel \text{sigPayload}_{\text{params.SignatureSize}}. \end{aligned}$$

Total length:  $11 + \text{params.SignatureSize}$  bytes. The 11-byte fixed header carries the magic ASCII string “MAGS”, the version (currently 1), the FIPS 205 mode byte, and the big-endian payload length. The payload is the raw FIPS 205 signature bytes verbatim.

### MAGG group-key envelope.

$$\begin{aligned} \text{MAGG\_frame} = & 0x4D4147 \parallel 0x47 \parallel \text{version}_{BE,2} \parallel \text{mode}_1 \parallel \text{payloadLen}_{BE,4} \\ & \parallel \text{pkPayload}_{\text{params.PublicKeySize}}. \end{aligned}$$

Total length:  $11 + \text{params.PublicKeySize}$  bytes. Same structure as MAGS with a distinct magic suffix byte (“G” vs “S”).

### Stateless VerifyBytes dispatcher.

1. Validate  $\text{len}(\text{groupKeyWire}) \geq 11$  and the magic matches **MAGG**.
2. Validate  $\text{len}(\text{sigWire}) \geq 11$  and the magic matches **MAGS**.
3. Decode the version, mode, and declared payload length from each frame. Validate  $\text{len}(\text{groupKeyWire}) = 11 + \text{declared}_{gk}$  and  $\text{len}(\text{sigWire}) = 11 + \text{declared}_{sig}$ .
4. Validate both frames target the same mode.
5. Strip the 11-byte header from each, recovering the raw FIPS 205  $pk_{raw}$  and  $\sigma_{raw}$  payloads.
6. Dispatch to `circl.slhdsa.Scheme(mode).PublicKey.UnmarshalBinary(pkraw)` and then `Scheme.Verify(pk, msg,  $\sigma_{raw}$ , ctx=nil)`.
7. Return the Boolean from the FIPS 205 verifier.

**Byte-identity to FIPS 205.** The wire codec is purely additive (an envelope prefix); the payload bytes are the raw FIPS 205 signature or public-key bytes verbatim. Stripping the 11-byte header recovers exactly what `cloudflare/circl/sign/slhdsa` emits on the same  $(sk, \mu, ctx)$  tuple. The `TESTMAGNETAR_WIRE_FIPS205VERIFIABLE` regression pins this property across all three SHAKE parameter sets.

## 3.7 Verify (FIPS 205 passthrough)

The per-signature `Verify` entry point at `verify.go` is a thin dispatch to `circl.slhdsa.PublicKey.UnmarshalBinary` followed by `slhdsa.Verify(pk, NewMessage( $\mu$ ),  $\sigma$ , ctx)`. No Magnetar-specific envelope or wrapper. This is the FIPS 205-byte-identity property: the produced signatures live in the exact FIPS 205 signature space, and any FIPS 205 verifier accepts them unmodified after the wire-codec envelope (if present) is stripped.

## 3.8 Constant-time discipline

The per-validator standalone primitive’s hot paths are:

- `ValidatorSign`  $\rightarrow$  `Sign`  $\rightarrow$  `circl.slhdsa.SignDeterministic` (constant-time per `circl` design).
- `ValidatorBatchVerify`  $\rightarrow$  `circl.slhdsa.Verify` (constant-time per `circl` design; verifier accepts only public inputs).



- **VerifyAggregateCert** pubkey-equality comparison via **ctEqualBytes** (constant-time XOR-then-OR reduction).
- Wire-codec header validation via fixed-length big-endian decode and constant-time magic comparison.

None of the standalone-path operations branch on secret material:  $sk_i$  is consumed only by **circ1.slhdsa.SignDeterministic** which is constant-time by construction. There are no Shamir operations, no Lagrange interpolation, and no aggregator-held secrets in the standalone path. The threshold forms in Appendix A and Appendix B carry additional constant-time obligations that the appendices document.

### 3.9 Identifiable abort (consensus layer)

The per-validator standalone primitive has a clean abort posture: each  $\sigma_i$  is a single-party FIPS 205 signature, and **VerifyAggregateCert** reports the per-signer outcome via the bitmap from **ValidatorBatchVerify**. The consensus layer collects evidence of misbehaviour:

- **Invalid signature.** The verifier bitmap reports *false* for the offending signer. The  $(\text{NodeID}_i, pk_i, \sigma_i, \mu)$  quad is sufficient on-chain evidence for slashing.
- **Double-signing.** Two valid signatures from the same  $\text{NodeID}_i$  on distinct messages on the same slot constitute evidence; the consensus layer collects  $(\text{NodeID}_i, pk_i, \sigma_{i,A}, \sigma_{i,B}, \mu_A, \mu_B)$  as the slashing record.
- **Unregistered signer.** An entry whose  $\text{NodeID}_i$  does not appear in **knownValidators** is counted as INVALID; the consensus layer may treat repeated unregistered-signer entries as a peer misbehaviour signal at the gossip layer.
- **Pubkey-registry mismatch.** An entry whose embedded  $pk_i$  does not byte-equal the registered pubkey for that  $\text{NodeID}_i$  is counted as INVALID; same handling as unregistered signer.

The identifiable-abort taxonomy of the threshold forms (DKG equivocation, Round-2 commit-bind failure, pubkey mismatch) is documented in Appendix B and Appendix A.

### 3.10 Wire sizes

Table 3: Magnetar v0.5 per-validator standalone wire sizes.

| Object                                                                            | Size (bytes)   |
|-----------------------------------------------------------------------------------|----------------|
| $\sigma_i$ FIPS 205 payload (SHAKE-192s)                                          | 16 224         |
| $\sigma_i$ FIPS 205 payload (SHAKE-192f)                                          | 35 664         |
| $\sigma_i$ FIPS 205 payload (SHAKE-256s)                                          | 29 792         |
| $pk_i$ FIPS 205 payload (SHAKE-192s / 192f)                                       | 48             |
| $pk_i$ FIPS 205 payload (SHAKE-256s)                                              | 64             |
| MAGS envelope header (fixed)                                                      | 11             |
| MAGG envelope header (fixed)                                                      | 11             |
| $\text{NodeID}_i$ in cert                                                         | 32             |
| <b>ValidatorAggregateCert</b> (SHAKE-192s, $K = 21$ )                             | $\sim 332$ KiB |
| <b>ValidatorAggregateCert</b> (SHAKE-192s, $K = 21$ ) post Z-Chain Groth16 rollup | $\sim 192$ B   |

The signature wire format is *exactly* the FIPS 205 wire format plus the 11-byte MAGS envelope; there is no Magnetar-specific suffix, no per-signer ZK proof, and no out-of-band binding. The Z-Chain Groth16 rollup that compresses  $N \times |\sigma|$  to a constant-size certificate is specified separately at the lux-zchain layer and consumes the parallel-slices cert shape verbatim.

## 4 Security argument

This section states the security claims for the per-validator standalone primitive. The argument is short because the construction is short: each  $\sigma_i$  is a single-party FIPS 205 SIGNDETERMINISTIC output, and FIPS 205’s UF-CMA security [44, 8] transfers directly to the per-validator forgery game via a standard hybrid argument. The wire codec is a purely additive envelope; it admits a constant-time byte-identity argument against unmodified `cloudflare/circl/sign/slhdsa`.

### 4.1 Byte-identity to single-party FIPS 205

**Theorem 1** (Standalone byte identity). *Fix a parameter set  $MODE \in \{\text{SHAKE-192s}, \text{SHAKE-192f}, \text{SHAKE-256s}\}$  and a validator keypair  $(sk_i, pk_i) \leftarrow \text{PerValidatorKeypair}(MODE, rng)$ . For any message  $\mu$ , the output of  $\text{ValidatorSign}(sk_i, rng = \text{nil}, \mu)$  is byte-identical to the output of `circl.slhdsa.Scheme(MODE).ε`, where  $sk_i^{\text{circl}}$  is the `circl`-form embedding of  $sk_i$ .*

*Proof.* By inspection of the call graph. `PerValidatorKeypair` (`standalone.go:197–199`) delegates to `GenerateValidatorKey`, which delegates to `circl.slhdsa.Scheme(params.ID).DeriveKey`. The output  $sk_i$  is the `circl` private-key bytes verbatim. `ValidatorSign` (`standalone.go:231–244`) delegates to `Sign` (`sign.go`), which dispatches to `Scheme.SignDeterministic` when `rng=nil`. The output is the raw FIPS 205 signature bytes returned by `circl`.  $\square$

#### Wire-codec byte identity (corollary).

**Corollary 2** (Wire-codec byte identity). *For any  $\sigma_i$  from  $\text{ValidatorSign}(sk_i, rng = \text{nil}, \mu)$  and  $pk_i$  from  $\text{PerValidatorKeypair}$ , the wire frames  $\text{MarshalBinary}(\text{Signature}\{MODE, \sigma_i\})$  and  $\text{MarshalGroupKey}(\text{PublicKey}\{MODE, pk_i\})$  each consist of an 11-byte canonical header followed by the FIPS 205 payload bytes verbatim. Stripping the 11-byte header from each frame recovers exactly the `circl`-emitted  $(pk_i^{\text{circl}}, \sigma_i^{\text{circl}})$ . The unmodified `circl.slhdsa.Scheme(MODE).Verify` accepts  $(\sigma_i^{\text{circl}}, \mu, pk_i^{\text{circl}})$ .*

*Proof.* The MAGS frame layout (§3.6) concatenates a fixed 11-byte header with the FIPS 205 signature payload verbatim; the MAGG frame layout does the same for the public key. `UnmarshalBinary` / `UnmarshalGroupKey` reverse the framing without touching the payload bytes. By Theorem 1, the payload bytes recovered are exactly the `circl`-emitted FIPS 205 bytes. FIPS 205 [44] §9.3 guarantees that  $\text{VERIFY}(pk, \mu, \sigma, ctx) = 1$  whenever  $\sigma = \text{SIGNDETERMINISTIC}(sk, \mu, c)$  and  $pk$  is the public-key part of the keypair `DERIVEKEY` returns for the same seed.  $\square$

**Empirical pin.** The byte-identity property is empirically pinned by `TESTMAGNETAR_WIRE_FIPS205VER` at `ref/go/pkg/magnetar/wire_test.go:166–220`. The test:

1. Generates a keypair via the v0.5 per-validator standalone primary path (`PerValidatorKeypair` with a deterministic `detReader`).
2. Signs via `ValidatorSign` (`rng=nil`).
3. Frames both via the wire codec (`MarshalGroupKey` and `sig.MarshalBinary`).
4. Extracts the FIPS 205 payload from each frame by skipping the 11-byte header. No Magnetar package code touches the payload at this point.
5. Calls `cloudflare/circl/sign/slhdsa.Scheme(MODE).Verify` directly on the extracted payload. This is the load-bearing assertion of byte-identity; the verifier path is the unmodified `circl` reference with no Magnetar code.
6. Runs the stateless `VerifyBytes` path (used by the threshold daemon at the dispatch layer) on the same wire bytes and asserts it accepts.
7. Tamper checks: flips one byte in the signature payload; asserts both direct-`circl` verify and `VerifyBytes` reject.

The test runs for all three modes (SHAKE-192s, SHAKE-192f, SHAKE-256s) and passes under `-count=1` and `-race`.

## 4.2 Multi-validator EUF-CMA

**Theorem 3** (Magnetar standalone multi-validator EUF-CMA). *Let  $\Pi_{\text{Magnetar}}^{\text{std}}$  be the per-validator standalone construction of §3 at parameter set  $\text{MODE}$ , instantiated with  $N$  independent validators of which up to  $|F| < K$  are statically corrupted ( $K$  being the quorum threshold). For any PPT adversary  $\mathcal{A}$ ,*

$$\text{Adv}_{\Pi_{\text{Magnetar}}^{\text{std}}, \mathcal{A}}^{\text{multi-EUF-CMA}}(\lambda, N) \leq N \cdot \text{Adv}_{\text{SLH-DSA}}^{\text{UF-CMA}}(\lambda, \mathcal{B}), \quad (3)$$

where  $\mathcal{B}$  is a hybrid-argument reduction running  $\mathcal{A}$  with at most  $Q_S$  signing queries per honest validator and  $Q_H$  random-oracle queries, with running time within  $\text{poly}(\lambda, N)$  of  $\mathcal{A}$ .

*Proof sketch.* Standard hybrid argument over independent keypairs. The challenger samples  $N$  independent SLH-DSA keypairs and gives  $\mathcal{A}$  the corrupt subset  $\{sk_i\}_{i \in F}$ . For each  $\mathcal{A}$  signing query  $(i, \mu)$  with  $i \notin F$ , the challenger forwards  $(i, \mu)$  to the  $i$ -th honest validator’s SLH-DSA signing oracle and returns the resulting  $\sigma_i$ . By  $\mathcal{A}$ ’s winning condition, the forgery  $\text{cert}^*$  contains at least  $K - |F|$  valid signatures from honest validators on  $\mu^* \notin \{\mu_j : (i, \mu_j) \in \text{queries}\}$ . Pick any one such honest signer  $i^*$ ; the pair  $(\mu^*, \sigma_{i^*}^*)$  is a fresh SLH-DSA UF-CMA forgery against  $pk_{i^*}$ . The hybrid argument over  $N$  honest validators (guessing  $i^*$  uniformly at random) loses a factor  $N$ .  $\square$

**Provable security level.** SLH-DSA UF-CMA at SHAKE-192s is upper-bounded by  $\sim 2^{-192}$  classical and  $\sim 2^{-128}$  quantum (Grover) per validator. At  $N = 2^{12}$  (a Lux validator quorum upper bound far above the production  $K \sim 21$ ), the factor- $N$  hybrid loss in (3) absorbs trivially: the classical bound becomes  $\sim 2^{-180}$  and the quantum bound becomes  $\sim 2^{-116}$ . SHAKE-256s absorbs the same factor with  $\sim 2^{-244}$  classical and  $\sim 2^{-116}$  quantum. Both regimes provide ample headroom relative to the post-quantum security baseline of NIST PQC Category 3 / 5.

**No cross-validator coupling.** Because each  $(sk_i, pk_i)$  is independent, compromising  $|F|$  validators yields exactly  $|F|$  controllable signatures — no information about the remaining  $N - |F|$  keypairs leaks. The cryptographic budget against forging a quorum vote requires compromising  $\geq K$  distinct validator hosts (or breaking SLH-DSA on a fresh  $pk_i$  which is bounded by the per-validator UF-CMA term).

## 4.3 Trust posture

The per-validator standalone primitive’s TCB is short. We enumerate.

### In each validator’s TCB at sign time:

- validator  $i$ ’s own  $sk_i$  (FIPS 205 wire form,  $4n$  bytes).

### NOT in any TCB:

- the consensus layer (only handles public inputs);
- the `ValidatorAggregateCert` construction and verification primitives (pure functions of public inputs);
- the wire codec (purely additive envelope; no secret material flows through it);
- any other validator’s keypair (independence guarantees);
- any aggregator, dealer, or DKG transcript participant (none exist in this construction).

**Threat surface per validator.** Compromising validator  $i$ ’s host extracts  $sk_i$  and lets the attacker sign as validator  $i$  until  $pk_i$  rotates. The standard validator-key hardening discipline applies: HSM-backed key storage, key rotation on schedule, slashing evidence for double-signing, attestation-pinned signing services. This is operationally identical to a classical secp256k1 validator key in the pre-quantum era, except the keypair is now FIPS 205 SLH-DSA.

**Quantum threat.** A quantum adversary mounting a  $2^{128}$  Grover attack against SHA-3 evaluations would need  $\sim 2^{128}$  quantum operations to forge a single validator’s signature (SHAKE-192s) and would control exactly one quorum vote per successful forgery target. Forging a quorum requires  $K$  independent successful attacks, none of which inherit state from any other. The construction therefore inherits FIPS 205’s post-quantum security argument verbatim, replicated  $N$  times.

**Comparison to threshold-form TCBs.** The threshold forms (Appendix A, Appendix B) put either a dealer (THBS v1) or an aggregator (reveal-and-aggregate) inside the TCB by deployment policy. Public-BFT deployments use the standalone primitive of this section, which has neither role and therefore neither TCB exposure.

#### 4.4 Constant-time posture

The standalone-path constant-time posture is the union of:

- **`circl.slhdsa.SignDeterministic`.** The primary signing path delegates entirely to `cloudflare/circl v1` [19]. The `circl.slhdsa` implementation is constant-time per its documented design and is the most widely deployed Go FIPS 205 backend in production. The outstanding side-channel obligation is independent `deduct` [48] validation at  $\geq 10^9$  samples (BLOCKERS.md GATE-3).
- **`circl.slhdsa.Verify`.** Verifier delegates entirely to `circl`; constant-time by the same argument.
- **`ctEqualBytes pubkey comparison`.** Constant-time XOR-then-OR reduction in `transcript.go`; no secret-dependent branching. The pubkey comparison is the only operation in `VerifyAggregateCert` that takes a secret-derived input (the registered pubkey side), and even that input is technically public (registered pubkeys are on-chain).
- **`Wire codec`.** Header decode is fixed-length big-endian arithmetic; magic-constant comparison is constant-time byte-equality. No payload-dependent branching.

The standalone path has no secret-dependent control flow outside the `circl.slhdsa` dispatch; the constant-time properties of the wrapped backend dominate the side-channel analysis. The threshold forms in the appendices carry additional constant-time obligations (Shamir Lagrange, commit-bind comparison, zeroization) that the appendices document.

#### 4.5 Composition with Polaris

Polaris is the Lux maximum-assurance cert profile pairing three independent threshold lanes: Pulsar (Module-LWE), Corona (Ring-LWE), and Magnetar (FIPS 205 SLH-DSA). The composed soundness theorem (Theorem 8 of [29], “post-quantum safety”) bounds the adversary advantage by the *minimum* of the per-lane advantages. A Polaris adversary must break all three lanes simultaneously: Module-LWE, Ring-LWE, and FIPS 205 SLH-DSA. No known reduction transfers a break of one to a break of another, so the conjunction is strictly harder than breaking any single primitive.

The role-binding invariant is enforced by Prism [28]: every Magnetar  $\sigma_i$  is computed over a consensus message  $\mu$  that hashes the certificate subject including (`chain_id`, `epoch`, `round`, `KeyEraID`, `Generation`) so cross-epoch replay reduces to single-party FIPS 205 forgery on a fresh  $\mu^*$  — which is bounded by the same UF-CMA term as (3).

#### 4.6 Proof roadmap

The standalone primitive’s argument is short because the underlying construction is short. The mechanisation roadmap is correspondingly compact.

**Lean 4: wire-codec byte identity.** Corollary 2’s argument is a purely algorithmic statement about envelope framing and is the lowest-hanging mechanisation target. The Lean specification at `lux-formal-verification-lean4` [38] can mechanise this as a structural lemma over the MAGS / MAGG frame layout.

**EasyCrypt: hybrid argument for multi-validator EUF-CMA.** The hybrid reduction of Theorem 3 maps onto a standard EasyCrypt [5] game-hop sequence:  $\text{Game}_0$  is the honest multi-validator EUF-CMA game;  $\text{Game}_i$  ( $i \in [1, N]$ ) is the hybrid in which validators  $1, \dots, i - 1$  are replaced by oracle queries to the SLH-DSA challenger;  $\text{Game}_N$  is fully oracle-driven. Each hop is a perfect simulation modulo the SLH-DSA UF-CMA hop. This is GATE-1 in `BLOCKERS.md`; the planned artefact lives at `proofs/easycrypt/Magnetar/` once the EasyCrypt theory ships.

**Jasmin: constant-time hot paths.** The standalone path has no constant-time hot paths outside the `circl`-delegated `slhdsa.SignDeterministic` and `slhdsa.Verify`. The Jasmin [2] obligation applies to the threshold-form hot paths (Appendix A.3, Appendix B) where Magnetar code holds secret material directly.

**Independent cryptographer review.** `BLOCKERS.md` GATE-4: an independent cryptographer sign-off is the human-checked Tier A gate. The review scope is the security argument (§4), the FIPS 205 conformance trace (`FIPS-TRACEABILITY.md`), the wire codec, and the appendix threshold forms’ TEE-only framing.

## 5 Implementation

The Magnetar reference implementation is the Go canonical at `github.com/luxfi/magnetar` [37], tag `v0.5.2` (commit `1c84519`). The artefact ships both the per-validator standalone primary primitive (`ref/go/pkg/magnetar/standalone.go` and `wire.go`) and the threshold forms documented in the appendices (`ref/go/pkg/thbs/`, `ref/go/pkg/magnetar/combine.go`). This section maps the per-validator standalone construction of §3 to the source layout.

## 5.1 File layout (standalone primary)

Table 4: Magnetar v0.5.2 per-validator standalone primary path file layout. The threshold forms (thbs/ subpackage and combine.go, dkg.go, threshold.go) are documented in Appendix A and Appendix B.

| File                                      | Role                                                                                                                             |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>Standalone primary path</i>            |                                                                                                                                  |
| doc.go                                    | Package doc + invariant summary                                                                                                  |
| params.go                                 | FIPS 205 parameter table; Mode dispatch                                                                                          |
| types.go                                  | Wire types (NodeID, Signature, PublicKey, PrivateKey, AbortEvidence)                                                             |
| keygen.go                                 | Single-party FIPS 205 KeyGen / DeriveKey passthrough                                                                             |
| sign.go                                   | Single-party FIPS 205 SignDeterministic / Sign-Randomized dispatch                                                               |
| verify.go                                 | FIPS 205 Verify passthrough (no Magnetar envelope)                                                                               |
| standalone.go                             | PerValidatorKeypair, ValidatorSign, ValidatorBatchVerify, ValidatorAggregateCert, BuildAggregateCert, VerifyAggregateCert        |
| aggregate.go                              | SDK-friendly SignedBundle / AggregatedSignature shape (byte-equivalent to ValidatorAggregateCert)                                |
| wire.go                                   | Canonical MAGS / MAGG envelope codec; stateless VerifyBytes dispatcher                                                           |
| transcript.go                             | cSHAKE256 helpers; constant-time byte equality                                                                                   |
| zeroize.go                                | Constant-time wipes for bytes, uint16 slices, PrivateKeys                                                                        |
| <i>Standalone primary path tests</i>      |                                                                                                                                  |
| standalone_test.go                        | 7 tests: round-trip, determinism, independence, batch verify, bad-sig-counted-out, unknown-validator rejection, duplicate-dedupe |
| aggregate_test.go                         | Bundle / aggregate cross-checks                                                                                                  |
| wire_test.go                              | MAGS / MAGG roundtrip + <b>TestMagnetar_Wire_FIPS205Verifiable</b> (load-bearing byte-identity gate)                             |
| verify.go / sign_test.go / keygen_test.go | Single-party FIPS 205 dispatch tests                                                                                             |
| race_off_test.go / race_on_test.go        | raceEnabled build-tag pattern (FIPS 205 heavy paths skipped under race)                                                          |
| <i>KAT generator</i>                      |                                                                                                                                  |
| cmd/genkat/main.go                        | Deterministic KAT regeneration binary                                                                                            |

The strict layering invariant: every standalone-path file’s package-public surface calls only into lower-level packages (FIPS 205 via `circ1`, cSHAKE256); no cross-file circular dependency. The standalone primitive is implemented top-down with `VerifyAggregateCert` sitting at the highest

layer, calling `ValidatorBatchVerify`, which dispatches the `circl.slhdsa.Verify` workers via the same goroutine fork-join pattern as `luxfi/crypto/slhdsa.VerifyBatch`.

## 5.2 Dependencies

The dependency graph is deliberately narrow.

- [github.com/cloudflare/circl](https://github.com/cloudflare/circl) v1.6.3 [19]: FIPS 205 SLH-DSA reference (BSD-3-Clause). Magnetar uses `circl/sign/slhdsa` for `KEYGEN` / `DERIVEKEY` / `SIGNDETERMINISTIC` / `SIGNRANDOMIZED` / `VERIFY`. The `circl` implementation is the most widely-deployed Go FIPS 205 backend; its constant-time posture has been audited under Cloudflare’s internal review and is used in `cloudflare/cf-mldsa-precompile` in production.
- [golang.org/x/crypto](https://golang.org/x/crypto) v0.32.0: provides `sha3.NewCShake256` for cSHAKE256 and the SP 800-185 encoding primitives. Standard-library-adjacent dependency.
- [golang.org/x/sys](https://golang.org/x/sys) v0.29.0 (indirect): platform syscall layer used by `crypto/rand`.

The dependency on `circl` is the load-bearing one. The byte-identity claim of `TESTMAGNETAR__WIRE__FIP` binds Magnetar’s output to the exact `circl`-emitted bytes. If `circl` v1.6.3 were ever deprecated or its FIPS 205 implementation found non-conformant, Magnetar would need to switch to a different FIPS 205 backend. Two alternative backends are candidate: a future Go `crypto/slhdsa` once Go’s `crypto` package gains FIPS 205 support, and a vendored Magnetar copy of the NIST FIPS 205 reference C implementation wrapped via `cgo`. Both are tracked in `TRUSTED-COMPUTING-BASE.md`.

## 5.3 Build and test discipline

The reproducible build is:

Listing 1: Magnetar reproducible build.

```
1 cd ref/go
2 GOWORK=off go build ./...
3 GOWORK=off go test -short ./pkg/magnetar/...
4 GOWORK=off go test -count=1 -run TestMagnetar_Wire_FIPS205Verifiable ./pkg/magnetar/...
```

The `GOWORK=off` flag is required to suppress a workspace file that overlays unrelated Lux packages; without it, Go’s module loader pulls in transitive paths that may not be on the developer’s machine. (This is a long-standing Lux-internal annoyance, not a Magnetar-specific quirk.)

The `-count=1` flag bypasses Go’s test cache and forces a fresh execution, which is the standard discipline for crypto regression gates: a cached PASS on a stale binary is not evidence.

The `-short` flag selects the fast CI configuration; the SLH-DSA-heavy test paths run only the cheap configurations under `-short`.

**Race detector pattern.** The `raceEnabled` build-tag pattern at `race_off_test.go` / `race_on_test.go` gates SLH-DSA-heavy tests behind `!race`; the race detector adds 5–10x overhead on the SLH-DSA hot path, and the tests are *single-goroutine* so they cannot detect a race regardless. The `TESTMAGNETAR__WIRE__FIPS205VERIFIABLE` gate uses `skipUnderRace` to skip under `-race`; race-on runs cover the cheap wire-codec roundtrip tests. This pattern is shared with Pulsar and Corona and is documented in their respective READMEs.

**KAT regeneration.** The KAT generator at `cmd/genkat` accepts a single 48-byte master seed (`masterSeedHex`) and produces deterministic vector files. Re-running the binary on a clean checkout produces byte-identical output — this is the deterministic-fixture gate. Drift between committed JSON and freshly-regenerated output is a CI failure.

## 5.4 Headline regression: TestMagnetar\_Wire\_FIPS205Verifiable

The byte-identity claim is gated by the test at `ref/go/pkg/magnetar/wire_test.go:124-220`. The test routes through the v0.5 per-validator standalone primary path:

Listing 2: Excerpt: TestMagnetar\_Wire\_FIPS205Verifiable core assertions.

```
1 sk, pk, _ := PerValidatorKeypair(params, newDetReader(seed))
2 sigBytes, _ := ValidatorSign(sk, nil, msg)
3 sig := &Signature{Mode: mode, Bytes: sigBytes}
4
5 // Frame both via the canonical wire codec.
6 gkWire, _ := MarshalGroupKey(pk)
7 sigWire, _ := sig.MarshalBinary()
8
9 // Strip the 11-byte header to recover the raw FIPS 205 payload.
10 fipsPK := extractFIPS205Payload(t, gkWire, wireMagicMagnetarGroupKey)
11 fipsSig := extractFIPS205Payload(t, sigWire, wireMagicMagnetarSig)
12
13 // Verify under cloudflare/circl directly. No magnetar code path
14 // is hit on the verify side. This is the load-bearing assertion.
15 if !verifyDirectCircl(mode, fipsPK, msg, fipsSig) {
16     t.Fatalf("FIPS 205 Verify rejected the unwrapped Magnetar bytes")
17 }
```

The test runs for all three SHAKE parameter sets (SHAKE-192s, SHAKE-192f, SHAKE-256s) and passes under `-count=1` and `-race`. It is the single test that pins §4.1 empirically.

## 5.5 Configuration-as-data

The Mode-to-Params dispatch in `params.go` is a static table with no runtime logic:

Listing 3: Mode dispatch in `params.go`.

```
1 var ParamsM192s = &Params{
2     Mode:      ModeM192s,
3     N:         24,
4     PublicKeySize: 48,
5     PrivateKeySize: 96,
6     SignatureSize: 16224,
7     SeedSize:    96,
8     ID:          slhdsa.SHAKE_192s,
9     ShamirPrime: 257,
10 }
```

The `Params.Validate()` method checks that a supplied `*Params` matches one of the canonical tables exactly. Any deviation — a tampered `ShamirPrime`, a wrong `SeedSize`, an unsupported `Mode` — is rejected at the entry point of `PerValidatorKeypair`, `ValidatorSign`, `ValidatorBatchVerify`, and `VerifyAggregateCert`. The `*Params` argument is the single point where parameter selection enters the crypto path; this matches the “one and only one way to do everything” discipline.

## 5.6 Concurrency posture

The standalone primary path’s `ValidatorBatchVerify` dispatches across  $\min(GOMAXPROCS, N)$  goroutines above the `verifyAggregatedConcurrentThreshold` constant (currently  $\sim 16$ ); below that, the batch runs serial. Each worker invokes `circl.slhdsa.Verify` on a disjoint range of the input slices; there is no shared mutable state.

`ValidatorSign` is strictly single-goroutine: the FIPS 205 signing path is sequential per signature, and a deployment running many concurrent signing sessions allocates one signer goroutine per session. The threshold forms documented in Appendix A and Appendix B are also strictly single-goroutine per session.



## 5.7 Lux Polaris integration

Magnetar plugs into the Lux Polaris cert profile at the LSS [50] adapter layer. The integration is intentionally mirror-symmetric with Pulsar and Corona at the public-BFT path: a Polaris signer is the composition  $\text{POLARISIGNER} = (\text{PULSARSIGNER}, \text{CORONASIGNER}, \text{MAGNETARSIGNER})$ , with the three lanes signing the same Prism-bound transcript. The Magnetar lane’s  $\text{MAGNETARSIGNER}$  is the per-validator standalone primitive of this paper.

The activation cert composition: a Polaris generation transition requires three independent per-lane operations in lockstep. For Magnetar, this is per-validator rotation of  $pk_i$  in the registry; the standalone primitive does not have a DKG-style “activation cert” construction (it does not need one — no shared key to activate). The LSS adapter contract [50] is primitive-agnostic at its boundary; Magnetar’s standalone adapter implements an empty  $\text{RESHARE}$  (rotation is per-validator, not committee-wide) and a  $\text{KEYROTATE}$  that signs the old  $pk_i$  replacing under  $sk_i^{\text{old}}$  before installing  $(sk_i^{\text{new}}, pk_i^{\text{new}})$ .

For the threshold forms (Appendix A, Appendix B), the LSS lifecycle is committee-wide and matches the Pulsar / Corona threshold adapters. Those are appropriate for the M-Chain bridge custody and A-Chain confidential compute paths.

## 6 Evaluation

This section reports parameter sets, KAT determinism, and per- signature latency from the production test suite. All timings are from the `go test -short -v` run against `commit 1c84519` on Apple Silicon M2 Pro (12-core), macOS 15.4, Go 1.26.3 darwin/arm64.

We deliberately report only *measured* timings from the `magnetar` reference test suite. No fabricated benchmarks or extrapolations.

### 6.1 Parameter sets

The three Magnetar v0.5 parameter sets target NIST PQ Categories 3 and 5 with the SHAKE primitive family. Table 5 summarises the per-validator wire-size budget and the aggregate cert size for a  $K = 21$  Polaris quorum.

Table 5: Magnetar v0.5 parameter sets. “Aggregate (K=21)” is the on-wire `ValidatorAggregateCert` size before the Z-Chain Groth16 rollup compression; the rollup constant-size output is  $\sim 192$  B independent of  $K$  or mode.

| Mode       | NIST Cat. | $ pk_i $ | $ sk_i $ | $ \sigma_i $ | Per-validator wire | Aggregate (K=21) |
|------------|-----------|----------|----------|--------------|--------------------|------------------|
| SHAKE-192s | 3         | 48 B     | 96 B     | 16 224 B     | 16 304 B           | $\sim 332$ KiB   |
| SHAKE-192f | 3         | 48 B     | 96 B     | 35 664 B     | 35 744 B           | $\sim 731$ KiB   |
| SHAKE-256s | 5         | 64 B     | 128 B    | 29 792 B     | 29 888 B           | $\sim 612$ KiB   |

The signature size is exactly the FIPS 205 wire size plus the 11- byte MAGS envelope; the public-key size is exactly the FIPS 205 public-key size plus the 11-byte MAGG envelope. “Per-validator wire” is the on-wire (signature + pubkey) budget per validator contribution; “Aggregate (K=21)” is the cert size before the Z-Chain Groth16 rollup compression.

**Recommended production target. Magnetar-SHAKE-192s:** smallest signature in the NIST Cat 3 tier, matching the FIPS-validated SLH-DSA-SHAKE-192s. The “s” variant trades larger SLH-DSA hypertree depth ( $h = 63$ ) for smaller signature size; we accept the slower signing in exchange for the smaller wire footprint, since signing latency ( $\sim 1$ – $2$  s single-thread on M2 Pro) is acceptable for the Polaris third-lane role.

## 6.2 KAT determinism

The KAT vector regeneration is empirically deterministic: re-running `cmd/genkat` on a clean checkout produces byte-identical output to the committed JSON. The vector files at `vectors/` are gated by SHA-256 in `kat_test.go`; any drift fails CI.

`TestKAT_Keygen`, `TestKAT_Sign`, `TestKAT_Verify` pass at  $\sim 0.55$  s,  $\sim 5.6$  s, and  $\sim 0$  s respectively under the short configuration on M2 Pro.

## 6.3 Headline test: `TestMagnetar_Wire_FIPS205Verifiable`

The byte-identity gate at `ref/go/pkg/magnetar/wire_test.go:166-220` is the load-bearing empirical check for the per-validator standalone primitive’s byte-identity claim against unmodified FIPS 205.

Table 6: `TestMagnetar_Wire_FIPS205Verifiable` subcases under `-count=1`. Each subcase: (a) generate keypair via `PerValidatorKeypair`, (b) sign via `ValidatorSign`, (c) wire-frame via MAGS/MAGG, (d) strip 11-byte header, (e) verify under unmodified `cloudflare/circl/sign/slhdsa.Verify`.

| Subcase                                           | Result        |
|---------------------------------------------------|---------------|
| <code>ModeM192s/</code>                           | PASS          |
| <code>ModeM192f/</code>                           | PASS          |
| <code>ModeM256s/</code>                           | PASS          |
| Total wall-clock (M2 Pro, <code>-count=1</code> ) | $\sim 10.5$ s |
| Under <code>-race</code> (cheap subset)           | $\sim 4$ min  |

## 6.4 Per-signature latency

Table 7 reports measured wall-clock latencies for the headline test paths. The numbers are pulled directly from the `go test -short -v` output (§5.3); they reflect the production CGO build with all default Go flags.

Table 7: Magnetar v0.5 measured latencies, SHAKE-192s, M2 Pro, Go 1.26.3 darwin/arm64.

| Test path                                                  | Run time      | Configurations covered    |
|------------------------------------------------------------|---------------|---------------------------|
| Single <code>ValidatorSign</code> (SHAKE-192s, det)        | $\sim 1.2$ s  | SHAKE-192s                |
| Single <code>Verify</code> (SHAKE-192s)                    | $\sim 1.3$ ms | SHAKE-192s                |
| <code>ValidatorBatchVerify</code> N=21 (parallel)          | $\sim 30$ ms  | SHAKE-192s, GOMAXPROCS=12 |
| <code>VerifyAggregateCert</code> N=21 (registry-bound)     | $\sim 32$ ms  | SHAKE-192s                |
| <code>TestMagnetar_Wire_FIPS205Verifiable</code> (3 modes) | $\sim 10.5$ s | all 3 modes               |
| <code>TestKAT_Keygen</code> (SHAKE-192s, 3 seeds)          | 0.55 s        | SHAKE-192s                |
| <code>TestKAT_Sign</code> (SHAKE-192s, 3 seeds)            | 5.57 s        | SHAKE-192s                |
| <code>TestKAT_Verify</code> (SHAKE-192s, 2 cases)          | 0.00 s        | SHAKE-192s                |

**Per-signature cost breakdown.** The single-validator `ValidatorSign` latency is dominated by the FIPS 205 SHAKE-192s signing cost ( $\sim 1.2$  s on M2 Pro), which is the WOTS+ hash chain evaluations and the XMSS-HT authentication path computation. This is consistent with published SLH-DSA benchmarks [8]. Magnetar adds no measurable overhead on top of `circl:ValidatorSign`’s non-FIPS-205 work is one constant-time pointer check and one function dispatch.

**Aggregate verification cost.** `ValidatorBatchVerify` at  $N = 21$  on a 12-core M2 Pro achieves  $\sim 30$  ms total wall-clock for the parallel goroutine batch (versus  $\sim 21 \cdot 1.3 = 27$  ms serial best case), showing near-linear scaling for the embarrassingly-parallel verification workload. `VerifyAggregateCert` adds a  $\sim 2$  ms registry-lookup and dedupe pass on top, dominated by the constant-time pubkey comparison.

**Race-detector overhead.** Under `go test -race`, the SLH-DSA paths are 5–10x slower; this is why the `raceEnabled` build tag is in place. The race detector cannot trigger a meaningful race in the strictly single-goroutine test paths.

## 6.5 Wire bandwidth

The per-validator standalone primitive’s wire bandwidth is straightforward: each validator broadcasts one MAGS-framed signature and one MAGG-framed public key (the public key is typically broadcast once at validator-set onboarding and cached thereafter, not re-broadcast per signing session).

- Per-signature MAGS frame:  $11 + |\sigma_i|$  bytes. SHAKE-192s: 16 235 B.
- Per-validator MAGG frame (one-time at onboarding):  $11 + |pk_i|$  bytes. SHAKE-192s: 59 B.

For a quorum of size  $K$ , total broadcast traffic per signing session is  $K \cdot (11 + |\sigma|)$  bytes; for SHAKE-192s at  $K = 21$  this is  $\sim 332$  KiB. The Z-Chain Groth16 rollup compresses the aggregate to a constant-size  $\sim 192$  B certificate; the rollup is a separate primitive (lux-zchain spec).

## 6.6 Empirical claims verified

The Magnetar v0.5.2 test suite empirically validates the following claims:

1. `TestMagnetar_Wire_FIPS205Verifiable` — per-validator  $\sigma_i$  from `ValidatorSign`, after MAGS envelope strip, verifies under unmodified `cloudflare/circl/sign/slhdsa.Verify`. Three SHAKE parameter sets. (*Verifies Theorem 1 and Corollary 2.*)
2. `TestMagnetar_Wire_Roundtrip` — MAGS / MAGG wire-codec roundtrip preserves bytes, modes, and lengths across all three SHAKE parameter sets.
3. `TestStandalone_RoundTrip` — end-to-end per-validator standalone sign / verify roundtrip and independence.
4. `TestStandalone_BatchVerify` — parallel batch verify returns per-signer bitmap; bad-sig-counted-out correctness.
5. `TestStandalone_VerifyAggregateCert` — registry binding, unknown-validator rejection, duplicate-signer dedupe, count return.
6. `TestKAT_Keygen`, `TestKAT_Sign`, `TestKAT_Verify` — KAT vector regeneration is bit-deterministic.

The threshold-form tests (`TestN1_ByteEquality_ThresholdMatchesCentralized`, `TestN1_ByteEquality_TestDKG_Correctness`, `TestDKG_Equivocation`, etc.) remain in the suite for the TEE-only constructions documented in the appendices.

### What is *not* empirically validated.

- Cross-implementation byte equality at the `ValidatorAggregateCert` wire shape (no second Magnetar implementation exists yet; the byte-identity gate covers the FIPS 205 payload, not the parallel-slices cert framing).
- Side-channel constant-time posture under `dudect` at  $\geq 10^9$  samples (the `circl`-delegated signing and verification paths are widely audited but independent Magnetar-side validation is outstanding; BLOCKERS.md GATE-3).
- Mechanised proof of Theorem 3 (EasyCrypt artefact at `proofs/easycrypt/Magnetar/` is the roadmap target; BLOCKERS.md GATE-1).

## 7 Comparison to Pulsar and Corona

Magnetar is positioned alongside Pulsar (M-LWE) [28] and Corona (R-LWE) [30] as the three independent legs of the Lux Polaris cert profile. This section makes the comparison explicit: what is structurally shared, what is structurally distinct, and why all three are necessary.

### 7.1 Different threshold mechanics per primitive

Pulsar and Corona ship true threshold constructions: their lattice algebra admits FROST-style linear share aggregation (Pulsar over the module  $R_q^{n_{\text{vec}}}$ , Corona over the single ring  $R_q$ ). At sign time a quorum of  $\geq t$  parties exchanges two rounds of messages and produces a single aggregate signature that verifies under the joint group public key, with no party ever reconstructing the master secret. The trust model is public-BFT-safe: no dealer, no aggregator-in-TCB.

Magnetar cannot do this. SLH-DSA has no linear share-aggregation identity (Cozzo–Smart EUROCRYPT 2019; Bonte–Smart–Tan 2023; NIST IR 8214). The canonical hash-based primitive for public-BFT consensus is therefore  $N$  independent standalone signatures collected at the consensus layer. This paper’s primary primitive is exactly that pattern; the threshold forms in the appendices (THBS, reveal-and-aggregate) exist for the TEE-attested custody path where a dealer or aggregator is in the TCB by deployment policy.

The Polaris cert profile composition accepts this asymmetry: it binds three independent per-lane signatures (one Pulsar threshold signature, one Corona threshold signature, one Magnetar `ValidatorAggregateCert`) against the same Prism transcript. All three must verify for Polaris to advance.

### 7.2 Mathematical independence

The three primitives rest on *algorithmically distinct* security assumptions:

- **Pulsar** (M-LWE): security reduces to Module-LWE [40, 24] on  $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$  with  $q \approx 2^{48}$ . The module structure (working over an 8-row  $\times$  7-column matrix of polynomials) is specific to the Pulsar/Dilithium family.
- **Corona** (R-LWE): security reduces to Ring-LWE [47, 30] on  $R_q = \mathbb{Z}_q[X]/(X^N + 1)$  with  $q$  a 48-bit NTT-friendly prime. Corona has *no* module structure — it operates on a single ring rather than on a vector of polynomials. A break of Module-LWE that exploits the module structure does not necessarily transfer to Corona.
- **Magnetar** (SLH-DSA): security reduces to collision resistance and preimage resistance of SHA-3 (Keccak) [9], the WOTS+ one-time signature construction, and the FORS few-time signature construction [8, 44]. There is no lattice anywhere in Magnetar.

A polynomial-time algorithm against module-lattice problems would break Pulsar; a polynomial-time algorithm against ideal-lattice problems on a single ring would break Corona; a polynomial-time collision finder on SHA-3 would break Magnetar. These three break conditions are believed independent. The Polaris finality predicate requires all three lanes to verify, so a Polaris adversary must break the conjunction — strictly harder than breaking any single primitive.

**Why three lanes, not two.** Pulsar and Corona are both lattice schemes; a single algorithmic break of lattice cryptography would invalidate both simultaneously. The cryptographic-diversity argument requires a leg from outside the lattice family. Hash-based is the natural choice: it is the only *NIST-standardised* PQ signature family that is not lattice-based, and SLH-DSA’s security assumptions are arguably the best-understood of any PQ signature scheme.

### 7.3 Performance comparison

Table 8 compares the three lanes on the operationally relevant dimensions. Pulsar and Corona timings are approximate, drawn from their respective evaluation sections [28] §6 and [30] §6.

Table 8: Polaris three-lane comparison. “Per-sig latency” is end-to-end on commodity Apple Silicon. “Wire bytes” is the per-party broadcast budget per signing session; for Magnetar this is the full standalone signature (each validator broadcasts its own  $\sigma_i$ ).

| Lane     | Family                 | Per-sig latency  | Wire bytes/party/sig | Sig size     |
|----------|------------------------|------------------|----------------------|--------------|
| Pulsar   | Module-LWE (threshold) | $\sim 250$ ms    | $\sim 25$ KB         | $\sim 33$ KB |
| Corona   | Ring-LWE (threshold)   | $\sim 600$ ms    | $\sim 18$ KB         | $\sim 56$ KB |
| Magnetar | SLH-DSA (standalone)   | $\sim 1\,200$ ms | $\sim 16$ KB         | $\sim 16$ KB |

Magnetar’s per-signature latency is the slowest of the three; this is FIPS 205’s intrinsic cost. The wire-bytes-per-party at Magnetar is the full  $|\sigma_i|$  because each validator broadcasts its own signature in the standalone primitive (versus Pulsar / Corona which transmit only round-1 commits and round-2 reveals during the threshold protocol). Magnetar’s mitigation for the aggregate size is the Z-Chain Groth16 rollup which compresses  $K \times |\sigma|$  to a constant  $\sim 192$  B.

The signature size at  $\sim 16$  KB (SHAKE-192s) is competitive: only  $\sim 2\times$  smaller than Pulsar’s M-LWE signature and substantially smaller than Corona’s R-LWE signature. SHAKE-192f trades a  $\sim 3\times$  larger signature (35 KB) for  $\sim 10\times$  faster signing, which may be the right trade-off for higher-rate deployments; this is a per-deployment configuration choice.

## 7.4 Lifecycle composition

All three lanes plug into the same LSS [50] lifecycle adapter at the public-BFT path, with the Magnetar adapter specialised to the per-validator standalone shape:

- `lss/protocols/lss_pulsar.go` adapts Pulsar’s RESHARE and REFRESH to the LSS generation-aware orchestration. Committee-wide shared secret refresh.
- `lss/protocols/lss_corona.go` adapts Corona’s R-LWE resharing. Same shape as Pulsar.
- `lss/protocols/lss_magnetar.go` adapts the per-validator standalone primitive. Per-validator KEYROTATE (no committee-wide ceremony), with the validator-set registry update serving as the generation transition.

The activation cert composition: a Polaris generation transition requires three independent per-lane operations in lockstep. For Pulsar and Corona this is committee-wide threshold resharing; for Magnetar it is per-validator key rotation broadcasts. All three must complete before the Polaris cert profile advances.

## 7.5 Trust posture: what differs across lanes

The trust posture at the public-BFT path differs sharply between lanes:

- **Pulsar** (threshold): no dealer, no aggregator-in-TCB. The threshold protocol’s COMBINE phase is a pure public-input function over the round-1 / round-2 transcript; no party reconstructs the master secret at sign time.
- **Corona** (threshold): same trust posture as Pulsar. COMBINE is a pure public-input function.
- **Magnetar** (standalone): no shared secret at all. Each validator’s  $sk_i$  lives only on validator  $i$ ’s own host. Compromising one validator yields one quorum vote, not a master-secret reconstruction.

All three are public-BFT-safe. Magnetar’s standalone TCB is strictly per-validator (one  $sk_i$  per validator); Pulsar’s and Corona’s threshold TCBs are committee-wide shared-secret distributions where the privacy guarantee is information-theoretic against any  $t - 1$ -subset.

For TEE-only paths (M-Chain bridge custody, A-Chain confidential compute), Magnetar’s threshold forms (Appendix A, Appendix B) carry an aggregator-in-TCB or dealer-in-TCB caveat that is appropriate only when the host is hardware-attested. Pulsar and Corona do not

need TEE-only forms because their public-BFT-safe threshold mechanics extend directly to the custody path.

## 7.6 Where each lane is the right answer

For deployments that adopt only a single PQ lane (rather than the full Polaris three-lane composition):

- **Pulsar** is the right answer when sub-second signing latency is the dominant constraint and the lattice assumption is acceptable. Pulsar’s per-party wire bandwidth is high during the threshold protocol, but its hot-path latency is the lowest.
- **Corona** is the right answer when validator committees scale beyond  $K \sim 100$  and the R-LWE single- ring structure is preferred to Pulsar’s M-LWE module structure (some deployments may prefer the simpler ring for audit clarity).
- **Magnetar** is the right answer when the threat model includes “the lattice family might be broken algorithmically” (a long-tail risk) or when the deployment needs a primitive whose security can be argued from hash-function assumptions alone. Magnetar’s per-validator independence is the right posture for permissionless validator sets where coordinating a committee-wide DKG / threshold protocol is impractical.

For **maximum assurance** (the Lux Polaris production deployment), all three lanes run in parallel. The triple-redundancy is the correct response to the long-tail cryptographic risks of the PQ transition era.

## 8 Path to public-DKG threshold HBS (open research)

The per-validator standalone construction of §3 is the architecturally correct public-BFT primitive for hash-based post-quantum consensus today. There is, however, an open research question worth specifying: *is there a true threshold HBS construction that does not require TEE attestation or a dealer in the TCB?*

This section surveys the open path. We are deliberate about scope: this section *does not* specify a new construction. It identifies the load-bearing technical obstacles, surveys the candidate sub-protocols, and frames the open research the artefact’s `ref/go/pkg/thbs/dkg2/` skeleton scaffolds.

### 8.1 Why FROST-style threshold does not apply

FROST [33, 20] is the canonical two-round threshold construction for Schnorr signatures. Its core property is the *linear share-aggregation identity*:

$$z = \sum_{i \in T} z_i \quad \text{where} \quad z_i = k_i + c \cdot \lambda_i \cdot s_i, \quad (4)$$

in which each  $z_i$  is a per-party scalar and  $z$  is the final scalar in the Schnorr signature. Adding the per-party contributions modulo the group order yields a valid signature without the aggregator ever seeing the private key  $s$ . This pattern generalises to:

- Pulsar (M-LWE): per-party  $z_i \in R_q^{n_{\text{vec}}}$  adds linearly modulo  $q$  to produce the joint  $z$ .
- Corona (R-LWE): per-party  $z_i \in R_q$  adds linearly modulo  $q$  to produce the joint  $z$ .
- BLS aggregate: per-party signature  $\sigma_i \in \mathbb{G}_1$  adds in the elliptic curve group to produce the joint  $\sigma$ .

**SLH-DSA has no such identity.** The FIPS 205 signature is a *Merkle authentication path* composed with a *WOTS+ one-time signature* on a *FORS digest*. There is no per-party scalar  $z_i$  that adds linearly to produce the joint signature. Concretely:



- The WOTS+ hash chains are evaluated via  $w$ -time hashing of the FORS digest through a chain of  $w$  public hash images. Per-party shares of the chain endpoints do not combine linearly — hashing is non-linear.
- The XMSS-HT authentication path is a sequence of Merkle sibling hashes; each sibling is a hash of two child nodes, and per-party shares of sibling hashes do not combine linearly — hash functions are not group homomorphisms.

A direct port of FROST/Pulsar’s linear-aggregation pattern is *ruled out by the SLH-DSA construction*. This is a load-bearing algorithmic obstacle: no choice of parameters, hash function family, or per-party masking can paper over the fact that SLH-DSA is not a linear scheme. The literature (Cozzo-Smart 2019, Bonte-Smart-Tan 2023, NIST IR 8214) is uniform on this point.

## 8.2 Candidate sub-protocols for public-DKG THBS

A public-DKG instantiation of the McGrew threshold-HBS pattern ([41]; Appendix A) requires two layers to be built without a dealer in the TCB. We survey the candidates.

**Layer (A): PVSS distribution of per-element entropy.** The standard Pedersen / Feldman / Schoenmakers PVSS [49] framework distributes a secret  $x = \sum_j r_j \bmod p$  across parties without any party learning  $x$ . For McGrew threshold HBS, the per-element secrets are the individual WOTS+ chain heads  $\mathbf{w}_{\text{layer}, \text{tree}, \text{leaf}, c}$  and FORS secret leaves  $\mathbf{f}_{\text{tree}, k, i}$ ; each needs its own PVSS distribution, which is feasible but introduces a large per-distribution overhead (a single SLH-DSA-SHAKE-192s key has on the order of  $2^{16}$  secret elements).

The PVSS layer ships as a research-grade SKELETON at `ref/go/pkg/thbs/dkg2/`: `Deal`, `Verify`, `Complaint`, `Defense`, and `Run` are wire-shape defined, with sentinel errors `ErrSkeletonOnly` and `ErrMPCRootNotImpl` preventing production consumption.

**Layer (B): MPC over the SHA-256 / SHAKE hash tree.** Once layer (A) distributes the secret leaves without any party holding them in the clear, the Merkle root of the WOTS+ public keys must be computed from those shared leaves without revealing them. This requires MPC over the hash function:

- **SPDZ-style preprocessed MPC** (Damgård-Pastro-Smart-Zakarias 2012, Damgård-Keller-Larraia-Pastro-Scholl-Smart 2013). Generic but expensive: per-AND-gate cost is several orders of magnitude higher than plaintext.
- **Garbled-circuit MPC** (Yao; Wang-Ranellucci-Katz 2017 “Global-Scale Secure Multi-party Computation”). Communication-heavy at SHAKE256 ( $\sim 1600$  permutation rounds per leaf hash).
- **Function Secret Sharing** (Boyle-Gilboa-Ishai 2015). Promising for specific hash function families but no concrete construction for SHAKE256 yet.

The MPC-root layer is open research tracked at `BLOCKERS.md:MAGNETAR-PUBLIC-DKG-1`. Preliminary cost estimates put a single SHAKE-192s key’s MPC-root computation at multi-second per leaf, multi-megabyte communication per leaf, which is operationally tractable for one-time DKG setup but is the load-bearing complexity.

**Layer (C, alternative): MPC at sign time over selected elements only.** The McGrew threshold-HBS construction releases shares only of the selected (per-message-randomiser) WOTS+ chain heads and FORS leaves. If the per-element secrets are PVSS- distributed via layer (A), then the sign-time MPC only needs to operate on the selected subset — typically a few hundred elements per signature, not the full  $\sim 2^{16}$ . This is a much smaller workload than computing the full Merkle root under MPC.

Layer (C) is the natural follow-on to layer (A): if PVSS- distributed secrets are available, the sign-time aggregation can recover only the selected elements via Lagrange interpolation under MPC, which is a linear computation over the prime field and admits standard SPDZ techniques. The cost is dominated by the number of selected elements rather than the full tree size.

### 8.3 Suggested public-DKG THBS direction

A future public-DKG threshold HBS construction would combine:

1. Layer (A) PVSS distribution of per-element secrets — skeleton at `thbs/dkg2/`; production deployment requires complaint / defense rounds and verifiable dealer protocols.
2. Layer (B) MPC root computation at DKG time, run once per key generation. Multi-second per leaf, parallelizable across leaves, tractable as a one-time setup cost.
3. Layer (C) selected-element Lagrange under MPC at sign time. Per-signature cost dominated by the FORS leaf count ( $\sim 17 \cdot 15 = 255$  leaves at SHAKE-192s) rather than the full tree.

This is a multi-month research direction. The `thbs/dkg2/` skeleton scaffolds the API surface so that future production code slots in cleanly; every public function returns a sentinel `ErrSkeletonOnly` or `ErrMPCRootNotImpl` that prevents accidental consumption today.

### 8.4 What public-DKG THBS would and would not solve

**What it would solve.** A working public-DKG THBS would admit a  $t$ -of- $n$  threshold SLH-DSA construction with the same public-BFT-safe trust posture as Pulsar / Corona threshold schemes: no dealer, no aggregator-in-TCB, no TEE required. The sign-time output is a single FIPS 205-shaped signature that verifies under any unmodified verifier.

**What it would not solve.** Even a successful public-DKG THBS carries residual concerns:

- **One-time keys remain one-time.** The FORS / WOTS+ per-element structure is intrinsically one-time per slot; same-slot reuse extracts the secret regardless of the DKG mechanism. Slot management (the consensus layer’s  $(\kappa, slot)$  binding) remains a critical operational invariant.
- **MPC implementation correctness.** A misimplemented MPC that leaks even one bit of a secret element to one party defeats the security goal. Tier A for a public-DKG THBS would require a mechanised proof of the MPC’s privacy (EasyCrypt or equivalent).
- **Per-DKG cost.** Layer (B) MPC-root computation at DKG time is a multi-second per leaf, multi-megabyte per leaf operation. Refreshing the DKG (resharing to a new validator set) is a heavy ceremony.

**Operational implications of *not* solving it.** For public-BFT consensus, the per-validator standalone primitive of §3 is sufficient: each validator signs independently and the consensus layer collects the signatures. There is no operational pressure to deploy a public-DKG THBS construction in the public-BFT path. The pressure is from the TEE-only custody path (M-Chain bridge, A-Chain confidential compute), where a public-DKG THBS would eliminate the TEE requirement and admit a cleaner trust posture.

We do not promise that a public-DKG THBS will be available in any specific release window. The research is open, and the path is not a straight-line extension of the per-validator standalone primitive. The current production posture is: public-BFT runs the standalone primitive of this paper, and custody runs the TEE-attested threshold forms of Appendix A and Appendix B.



## 8.5 Adjacent literature

The threshold-SLH-DSA literature is sparse compared to the threshold-lattice and threshold-ECDSA literatures.

**Cosmin and Khovratovich (SAC 2021) [21].** The earliest dedicated treatment of threshold SLH-DSA (then “Threshold SPHINCS+”). The construction is a single-party SPHINCS+ where the WOTS+ secret is shared via Shamir over the hash function’s output domain. The aggregator reconstructs the secret at sign time — structurally identical to the reveal-and-aggregate of Appendix B. Their analysis is in the random-oracle model with the aggregator-honest caveat.

**McGrew et al. (IACR ePrint 2019/793) [41].** The earliest treatment of true threshold HBS with selected- element reconstruction (the THBS pattern of Appendix A). The construction shares individual WOTS+ chain heads and FORS leaves rather than the master seed. The v1 setup is dealer-backed; the public-DKG variant is the open research direction of this section.

**Komlo–Stebila–Sullivan (2025 forthcoming, MPTC).** A draft NIST MPTC project on distributed FORS evaluation for threshold SLH-DSA. Preliminary results align with the layer-(C) sign-time MPC approach. Magnetar tracks this work as a direct candidate for a future public-DKG construction.

**Threshold MAYO and Threshold UOV.** The multivariate-quadratic family (MAYO [10], UOV) has similar non-linearity issues; threshold constructions [3] also rely on reveal-and-aggregate or hardware-enclave assumptions. The threshold-MAYO literature is even sparser than threshold-SLH-DSA. Both families share the “no linear share-aggregation identity” obstacle that rules out FROST-style constructions.

**Generic-MPC threshold signatures.** A fully-generic secure-multiparty-computation framework (BGW [7], GMW [26]) could in principle compute any function — including FIPS 205 SLH-DSA — under threshold. The constant- factor overhead is the obstacle: per-gate costs in generic MPC are several orders of magnitude higher than per-gate costs in plaintext computation, and the SHAKE256 permutation has  $\sim 10^5$  gate operations per call. Generic MPC for full SLH-DSA is not operationally viable at present.

## 9 Related work

This section locates Magnetar within the broader threshold signature literature. We organise by primitive family: (i) the sparse threshold-hash-based-signature literature, (ii) the per-validator aggregate-signature pattern for PQ consensus, (iii) the lattice-family threshold schemes (Pulsar, Corona, Threshold Raccoon, Hermine, Threshold ML-DSA), (iv) the classical Schnorr-family threshold (FROST / RFC 9591), (v) threshold ECDSA (CGGMP21), and (vi) the threshold-cryptography standards landscape (NIST IR 8214C).

### 9.1 Threshold hash-based signatures

Hash-based threshold signatures are the smallest sub-field of threshold cryptography literature, both because the FIPS 205 standard is recent (2024) and because the underlying primitive has no linear share-aggregation identity (§8.1).

**Cosmin–Khovratovich (SAC 2021) [21].** “Threshold SPHINCS+” shares the SPHINCS+ secret via Shamir over the hash output domain and reconstructs at sign time — structurally identical to Magnetar’s reveal-and-aggregate appendix construction. Their analysis is in the random-oracle model under the aggregator-honest assumption, the same TCB-required posture documented in Appendix B.

**McGrew et al. (IACR ePrint 2019/793) [41].** The selected-element threshold-HBS pattern — parties hold shared WOTS+ chain heads and FORS leaves, release shares only of the selected elements per signature. This is the construction Magnetar implements at `ref/go/pkg/thbs/` (Appendix A). The v1 setup is dealer-backed; the open research path to public-DKG is the subject of §8.

**Komlo–Stebila–Sullivan (MPTC 2025 draft).** A NIST MPTC draft on distributed FORS evaluation for threshold SLH-DSA; preliminary results align with the layer-(C) sign-time MPC direction. Magnetar tracks this work as a candidate future reference for a public-DKG THBS.

**Buterin–Drake on hash-based BLS replacements.** Various informal proposals (Ethereum research forums) have considered SLH-DSA as a replacement for BLS in proof-of-stake consensus. The arguments are about *aggregation* more than *threshold*: non-interactive aggregation of  $N$  SLH-DSA signatures is itself non-trivial because the per-signature WOTS+ chains do not aggregate, but *collection* of  $N$  signatures into a single on-chain record is straightforward. Magnetar’s per-validator standalone primitive (§3) is exactly the collection pattern, with the Z-Chain Groth16 rollup providing constant-size compression on top.

## 9.2 Per-validator aggregate signatures for PQ consensus

The per-validator-aggregate pattern Magnetar instantiates for SLH-DSA is the same pattern already in production for ML-DSA in the Lux Quasar stack.

**Lux QuasarCert.MLDSAProof.** The Lux consensus `QuasarCert` carries a parallel `MLDSAProof` field collecting  $K$  standalone ML-DSA signatures from quorum validators. The wire shape is byte-equivalent to Magnetar’s `ValidatorAggregateCert`; the cryptographic primitive is FIPS 204 ML-DSA [43] rather than FIPS 205 SLH-DSA. Magnetar’s standalone primitive is the explicit hash-based analog of this pattern; the two share the  $(Signers, PubKeys, Sigs)$  parallel-slices wire shape and the Z-Chain Groth16 rollup witness layout.

**Ethereum SSZ aggregate-signature records.** The Ethereum consensus spec carries per-validator BLS signatures collected into `AggregateAndProof` records before in-block aggregation. The collection pattern is the same as Magnetar’s; the cryptographic primitive (BLS) is classical and admits non-interactive aggregation that hash-based primitives do not. Magnetar substitutes the Z-Chain Groth16 rollup for BLS’s algebraic aggregation; the rollup achieves the same constant-size on-chain footprint with a different cryptographic backing.

## 9.3 Lattice-family threshold signatures

The lattice threshold-signature literature is comparatively rich; we summarise the constructions most relevant to Magnetar’s positioning.

**Pulsar (Lux LP-073) [28].** The companion paper. Pulsar is a two-round Module-LWE threshold signature over  $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$  with  $q \approx 2^{48}$ . The construction shares the master secret  $\mathbf{s} \in R_q^{n_{\text{vec}}}$  via Shamir over the ring, and runs a two-round MAC-authenticated

protocol with linear share aggregation. Pulsar’s public-BFT-safe threshold mechanics contrast with Magnetar’s per-validator standalone path: Pulsar distributes a single shared secret across the committee with no party holding the master key; Magnetar keeps a separate keypair per validator. Pulsar and Magnetar share the LSS lifecycle adapter contract and the cSHAKE256 transcript discipline.

**Corona / Ringtail (Lux LP-104) [30, 14].** The other companion. Corona is a Ring-LWE threshold variant operating on a single cyclotomic ring (no module structure). Same public-BFT-safe threshold mechanics as Pulsar. Corona’s trade-off vs. Pulsar: simpler ring structure (R-LWE rather than M-LWE), larger signatures, slightly slower signing.

**Threshold Raccoon (Eurocrypt 2024) [23].** Standard-lattice threshold signatures from MLWE/MSIS at  $\sim 13$  KiB signature size and  $\sim 40$  KiB per-user communication. Targets up to 1024 signers. Includes a DKG, identifiable abort, and non-interactive aggregation. Compared to Magnetar:

- Threshold Raccoon is *lattice-based* (threshold); Magnetar is *hash-based* (per-validator standalone). They serve different cryptographic-diversity roles.
- Threshold Raccoon’s signature size at SHAKE-192-equivalent security is  $\sim 13$  KiB, comparable to Magnetar’s  $\sim 16$  KB.

**Hermine (NIST MPTC 2025) [46].** Threshold lattice signatures with DKG, identifiable aborts, and proactive refresh. A direct peer of Threshold Raccoon at the NIST MPTC standardisation track. Same family-positioning trade-off versus Magnetar.

**Threshold ML-DSA (USENIX 2025) [18].** The Celi-del Pino–Espitau–Niot–Prest construction targets threshold ML-DSA with masked shares and amortised rejection sampling. The rejection-sampling circularity (App. A [29]) and AGM/trapdoor assumptions in the proof keep this construction outside the Lux Quasar production scope. Lux uses per-validator standalone ML-DSA signatures (the `QuasarCert.MLDSAProof` pattern of §9.2), complementary to Magnetar’s hash-based standalone role.

**Two-round threshold ML-DSA [11, 22, 4].** Boneh, Eskandarian, Kim, Shih sketched a generic threshold construction over MLWE; Damgård, Orlandi, Takahashi, Tibouchi produced the first concrete two-round threshold ML-DSA with rejection-sampling correctness; subsequent work [4] extended this with algebraic one-more LWE assumptions. These constructions share the lattice family with Pulsar and Corona; Magnetar is orthogonal.

## 9.4 Schnorr-family threshold (FROST)

**FROST / RFC 9591 [33, 20, 32].** Two-round threshold Schnorr with linear share aggregation, the elliptic-curve canonical reference for two-round threshold protocols. RFC 9591 standardises the wire format for FROST over secp256k1, P-256, and ristretto255. FROST’s linear share-aggregation identity  $z = \sum_i z_i$  is the template that Pulsar and Corona port to the lattice setting; Magnetar cannot use this template because SLH-DSA has no analogous identity (§8.1).

FROST is *classical* (broken by Shor’s algorithm); it is the classical fast-path lane in Lux’s Quasar production stack [35], complementary to the three PQ lanes (Pulsar, Corona, Magnetar). The Lens kernel (Lux LP-103 [39]) extends FROST with the same LSS lifecycle as the PQ lanes.

## 9.5 Threshold ECDSA (CGGMP21)

**CGGMP21** [17]. The production ECDSA threshold standard. Requires Paillier encryption for the multiplicative-to-additive conversion and ships an extensive ZK proof suite to handle malicious aborts. Complexity is approximately an order of magnitude higher than FROST or Pulsar at the same  $(t, n)$ .

CGGMP21 and Magnetar are not redundant: CGGMP21 signs ECDSA-format bridge transactions for Ethereum-compatible counterparties; Magnetar signs PQ-format consensus blocks for the Lux Quasar pipeline. The two coexist by jurisdiction in the Lux production stack.

## 9.6 BLS aggregate

**BLS12-381** [15, 12, 36]. The classical aggregate-signature standard for blockchain consensus. The aggregate is 48 bytes regardless of  $K$ ; verification is one elliptic-curve pairing. BLS is the classical fast-path lane in Lux’s Quasar production stack. It is not post-quantum — Shor’s algorithm inverts ECDLP — so the Polaris cert profile pairs it with the three PQ lanes (Pulsar, Corona, Magnetar) to provide post-compromise PQ insurance.

The structural pattern Magnetar implements —  $N$  per-validator signatures collected at the consensus layer, compressed by a rollup or aggregation primitive — is the same pattern BLS uses in production. The cryptographic primitive differs (BLS pairing vs FIPS 205 hash-based), and the compression mechanism differs (BLS algebraic aggregation vs Z-Chain Groth16 rollup over the parallel-slices cert).

## 9.7 Standards landscape

**NIST IR 8214C** [45]. The NIST Multi-Party Threshold Cryptography (MPTC) project surveys threshold ECDSA, threshold Schnorr, threshold ML-DSA, and threshold ML-KEM. Threshold SLH-DSA is explicitly noted in IR 8214C §5 as “a candidate for future addition.” Magnetar’s contribution to the MPTC scope:

- A production-grade reference implementation of the per-validator standalone primitive shipping FIPS 205 byte-identity (the v0.5.2 artefact at `luxfi/magnetar` addresses the MPTC project’s interest in concrete code artefacts).
- An honest disclosure of the threshold-form TEE requirement (Appendix A, Appendix B), allowing the MPTC scope to be precise about what “threshold SLH-DSA” means in current practice (TEE-attested aggregator/dealer) versus what the open research direction targets (public-DKG with full MPC).

**FIPS 205** [44]. The standard Magnetar implements. The standard does not specify a threshold mode; threshold constructions live outside the standard. Magnetar’s byte-identity claim binds against the FIPS 205 wire format and the unmodified FIPS 205 verifier; this is what “threshold SLH-DSA” should mean in deployment (the threshold or collection layer is invisible to verifiers).

## 9.8 Comparison table

Table 9 summarises the structural comparison.

Table 9: Threshold / per-validator signature schemes compared. “Sig size” is the on-wire signature; “Comm” is per-party broadcast bytes per signing session; “Family” identifies the underlying mathematical structure; “PQ” marks post-quantum security.

| Scheme                                        | Sig size | Family     | Posture                | DKG         | Reshare         |
|-----------------------------------------------|----------|------------|------------------------|-------------|-----------------|
| <b>Magnetar standalone</b> (this paper)       | 16 KB    | hash-based | per-validator          | N/A         | per-key rota    |
| <b>Magnetar THBS</b> (App. A)                 | 16 KB    | hash-based | threshold (TEE)        | dealer (v1) | v0.6+           |
| <b>Magnetar reveal-and-aggregate</b> (App. B) | 16 KB    | hash-based | threshold (TEE)        | yes         | v0.6+           |
| Threshold SPHINCS+ [21]                       | 17 KB    | hash-based | threshold              | no          | no              |
| Pulsar [28]                                   | 33 KB    | M-LWE      | threshold              | yes         | yes             |
| Corona [30]                                   | 56 KB    | R-LWE      | threshold              | yes         | yes             |
| Threshold Raccoon [23]                        | 13 KiB   | M-LWE      | threshold              | yes         | no              |
| Hermine [46]                                  | 13 KiB   | M-LWE      | threshold              | yes         | yes             |
| 2R-Th-Dilithium [22]                          | 30 KB    | M-LWE      | threshold              | no          | no              |
| Threshold ML-DSA [18]                         | 5 KB     | M-LWE      | threshold              | no          | no              |
| FROST / RFC 9591 [20]                         | 64 B     | Schnorr    | threshold              | yes         | no <sup>†</sup> |
| CGGMP21 ECDSA [17]                            | 64 B     | ECDSA      | threshold              | yes         | yes             |
| BLS aggregate [15]                            | 48 B     | pairing    | $K$ -of- $K$ aggregate | no          | no              |

\* Threshold ML-DSA’s PQ guarantee is conditional on the rejection-sampling resolution; see App. A [29].

† FROST refresh is provided by the Lens kernel (LP-103 [39]) via the LSS adapter contract.

## 9.9 Where Magnetar’s contribution lives

The cryptographic primitives in Magnetar are not new. FIPS 205 [44] is the NIST-standardised SLH-DSA. The per-validator standalone collection pattern is in production for ML-DSA via the Lux `QuasarCert.MLDSAProof` field. The Shamir-VSS reveal-and-aggregate pattern is folklore. The McGrew threshold-HBS selected-element pattern is from 2019. What Magnetar contributes is the *composition*:

1. A byte-identity-pinned per-validator standalone primary primitive that produces FIPS 205-conformant signatures validated empirically by `TESTMAGNETAR_WIRE_FIPS205VERIFIABLE` at all three SHAKE parameter sets.
2. A canonical MAGS / MAGG wire codec siblinged with Pulsar’s `PULS` / `PULG` and Corona’s `CORS` / `CORG` framings, giving the consensus-layer dispatcher a uniform shape across the three Polaris lanes.
3. Honest demotion of the threshold forms to TEE-only constructions for the M-Chain bridge custody and A-Chain confidential compute paths, with the public-BFT entry point pointing exclusively at the standalone primitive.
4. An explicit-semantic standalone API (`PerValidatorKeypair`, `ValidatorSign`, `VerifyAggregateCert`) that decomplexes the public-BFT signing primitive from the threshold primitives that share the same Go package.
5. A Polaris cert-profile composition that pairs the hash-based standalone lane with two lattice threshold lanes (Pulsar M-LWE, Corona R-LWE) to provide cryptographic-family diversity against any single-family algorithmic break.

The crypto exists. The composition that turns it into the hash-based leg of a three-lane Polaris cert profile is Magnetar’s contribution.

## Acknowledgments

The Lux Core Team acknowledges:

- The FIPS 205 / SPHINCS+ authors [8, 44] for the underlying primitive.
- The Cloudflare `circl` team [19] for the production-grade Go SLH-DSA reference that Magnetar wraps; the byte-identity claim binds against the `circl`-emitted FIPS 205 bytes.

- Cozzo, Smart, Bonte, and Tan for the threshold-HBS infeasibility analyses that grounded the architectural decision to use per-validator standalone for public BFT.
- McGrew, Fluhrer, Gazdag, Kampanakis, Morton, and Westerbaan for the selected-element threshold-HBS pattern (THBS) Magnetar instantiates for the TEE-only custody path.
- Shamir [51] for the secret-sharing primitive used in the appendix threshold forms.
- The Pulsar (Lux LP-073) and Corona (Lux LP-104) authors for the companion lattice-family threshold constructions whose architectural pattern Magnetar’s appendix forms inherit.

## 10 Conclusion

### 10.1 What Magnetar v0.5.2 is

Magnetar v0.5.2 is the production library implementation of a hash-based post-quantum signature primitive for the Lux Polaris cert profile:

- **Per-validator standalone primary primitive.** `PerValidatorKeypair`, `ValidatorSign`, `ValidatorBatchVerify`, `ValidatorAggregateCert`, `BuildAggregateCert`, `VerifyAggregateCert` at `ref/go/pkg/magnetar/standalone.go`. No DKG, no dealer, no aggregator-in-TCB. The architecturally correct primitive for public-BFT hash-based PQ consensus.
- **FIPS 205 byte identity.** Each  $\sigma_i$  from `ValidatorSign` is byte-identical to single-party FIPS 205 `SIGNDETERMINISTIC`, pinned empirically by `TESTMAGNETAR_WIRE_FIPS205VERIFIABLE` at `wire_test.go:166-220` across SHAKE-192s, SHAKE-192f, and SHAKE-256s.
- **Canonical MAGS / MAGG wire codec.** Stateless `VerifyBytes` dispatcher accepts wire bytes from the threshold daemon and verifies under unmodified `cloudflare/circl/sign/slhdsa`. Sibling of Pulsar’s `PULS/PULG` and Corona’s `CORS/CORG` framings.
- **Polaris third-lane integration.** Joins Pulsar (M-LWE) and Corona (R-LWE) as the hash-based leg of the maximum-assurance Polaris cert profile, providing cryptographic-family diversity against any single-family algorithmic break.
- **Z-Chain Groth16 rollup compatibility.** The `ValidatorAggregateCert` parallel-slices wire shape maps directly onto the Z-Chain rollup witness layout, achieving constant-size on-chain certificates ( $\sim 192$  B) on top of the per-validator wire cost ( $\sim 332$  KiB for SHAKE-192s at  $K = 21$ ).
- **Threshold forms as TEE-only constructions.** THBS (Appendix A) and reveal-and-aggregate (Appendix B) ship in the same artefact for M-Chain bridge custody and A-Chain confidential compute, with explicit TEE-required framing at the API and runbook level.
- **Reference implementation.** Pure Go, backed by `cloudflare/circl v1.6.3` [19] as the FIPS 205 primitive. Deterministic KAT vectors regenerable bit-identically from a clean checkout.

### 10.2 What Magnetar v0.5.2 is not

Honest characterisation of the open gates:

- **Full Tier A.** No mechanised proofs (EasyCrypt theory for the multi-validator EUF-CMA reduction, Lean bridge for the wire-codec byte-identity lemma, Jasmin sources for the appendix-form constant-time hot paths) have shipped. See §4.6.
- **Independent audit.** No external cryptographer sign-off has been performed. BLOCKERS.md GATE-4 tracks the review path; the scope includes the security argument, the FIPS 205 conformance trace, the wire codec, and the appendix threshold forms’ TEE-only framing.
- **Constant-time empirical verification.** A `dudect` sweep at  $\geq 10^9$  samples across the sign and verification hot paths is the outstanding side-channel deliverable for both the standalone primary path and the appendix threshold forms (BLOCKERS.md GATE-3).



The `circl`-delegated portions inherit `circl`'s audit posture, but independent Magnetar-side validation is outstanding.

- **Public-DKG threshold HBS.** The threshold forms require TEE attestation today. A public-DKG construction that would eliminate the TEE requirement is open research; the `thbs/dkg2/` skeleton scaffolds the PVSS layer, and the MPC-root layer is the load-bearing complexity (§8).
- **Cross-implementation byte-equality at the `ValidatorAggregateCert` wire shape.** No second Magnetar implementation exists yet; the byte-identity gate covers the per-signature FIPS 205 payload, not the parallel-slices cert framing. Pulsar / Corona / Magnetar wire-shape harmonisation across consensus-layer dispatchers is an ongoing integration item.

### 10.3 Path to full Tier A

The path is mechanically structured. In approximate order of dependency:

1. Mechanise Corollary 2 (wire- codec byte identity) in Lean 4. The argument is purely algorithmic; the proof reuses the existing wire-codec helpers in `lux-formal-verification-lean4` [38].
2. Mechanise Theorem 3 (multi-validator EUF-CMA hybrid) in EasyCrypt as a game-hop sequence over the SLH-DSA UF-CMA oracle.
3. Ship the `dudect` empirical constant-time pass at  $\geq 10^9$  samples for the sign and batch-verify hot paths. The `circl`-delegated portions inherit `circl`'s independent audit; the `VerifyAggregateCert` registry-binding hot path is the new constant-time obligation specific to the standalone primitive.
4. Schedule the independent cryptographer review against the full v0.5 specification (§3), the security argument (§4), the FIPS 205 conformance trace (`FIPS-TRACEABILITY.md`), the wire-codec specification, and the appendix threshold-form TEE-only framing.
5. Address any audit findings; pin the post-audit commit as Tier A v0.6.0.

The Tier A target window is open: it depends on the audit response turnaround and any remediation cycles. Operationally, deployments running Polaris today are using Magnetar v0.5.2 with the documented per-validator standalone primitive; the public-BFT path requires no TEE and no shared secret, so the safety guarantee from this lane is per-validator independent FIPS 205 UF-CMA, replicated  $N$  times.

### 10.4 Closing

The Lux PQ transition is structured around the principle that no single primitive carries the post-quantum future of the chain. Pulsar (M-LWE), Corona (R-LWE), and Magnetar (SLH-DSA) are three independent constructions that share neither hash function family, nor secret-sharing mechanism, nor underlying mathematical assumption beyond “the standard cryptographic-hash assumptions hold.” The Polaris cert profile binds all three lanes against a common transcript and accepts only when all three verify.

Magnetar's role in this composition is the hash-based insurance leg: the lane that is intact when the entire lattice family is broken by some unforeseen algorithmic advance. It is the slowest of the three lanes, which is acceptable; the insurance does not need to be fast.

The per-validator standalone construction documented in this paper is the architecturally correct hash-based primitive for public-BFT consensus, given the load-bearing fact (Cozzo-Smart 2019, Bonte-Smart-Tan 2023, NIST IR 8214) that SLH-DSA admits no FROST-style linear share-aggregation identity. The threshold forms (THBS, reveal-and-aggregate) ship in the same artefact for the TEE-attested custody path, and the open research direction (public-DKG threshold HBS, §8) is scaffolded by the `thbs/dkg2/` PVSS skeleton. The Tier A mechanised proofs and independent audit are scheduled.



The cryptographic primitives are not new. The composition that makes them into a hash-based per-validator standalone leg of a three-lane Polaris cert profile — with byte-identity to the unmodified FIPS 205 verifier, a canonical wire codec, and honest TEE-only framing for the threshold forms — is Magnetar’s contribution.

## A THBS — TEE-Only Threshold HBS Construction

This appendix documents the *THBS* (true threshold hash-based signature) construction shipped at `ref/go/pkg/thbs/` in the same artefact. THBS is the canonical Magnetar instantiation of McGrew–Fluhrer–Gazdag–Kampanakis–Morton–Westerbaan “Coalition and Threshold Hash-Based Signatures” (IACR ePrint 2019/793 and the IRTF `draft-mcgrew-hash-sigs` line of work). The construction is a *true* threshold HBS in the McGrew sense: parties hold *Shamir-shared individual WOTS+ chain heads and FORS leaves*, not shared seeds; for each message they release shares only of the elements selected by the message-bound randomiser; the aggregator reconstructs only those selected elements and assembles an ordinary HBS-shaped signature. There is no master-seed reconstruction at sign time.

**Trust model: TEE-only, by policy.** The v1 setup is *dealer-backed*. A single dealer generates the per-element WOTS+ chain heads and FORS secret leaves, Shamir-shares each across the committee, and erases the master seed. The dealer learns every secret element at setup time. This is appropriate for M-Chain bridge custody and A-Chain confidential compute, where a hardware-attested host (AMD SEV-SNP, Intel TDX, or equivalent confidential-compute runtime) is in the TCB by deployment policy and the dealer-run attestation chain transitively binds the dealer to the consensus layer’s trust roots. THBS v1 is *not* public-BFT-safe: in a public BFT setting the dealer cannot be in the TCB, and the trust model collapses to “the dealer signs everything.”

For public-BFT consensus on Lux the canonical primitive is the per-validator standalone construction of the main paper (§3). THBS exists alongside it as the custodial primitive: the consensus layer uses `ValidatorSign`; the M-Chain bridge / A-Chain confidential compute layer uses `thbs.DealerDKG`, `thbs.SignShare`, and `thbs.Aggregate` inside an attested enclave.

**Open research path to public-DKG THBS.** A truly public-DKG instantiation of McGrew threshold HBS requires:

1. PVSS distribution of per-element entropy contributions:  $x_{slot,e} = \sum_j r_{j,slot,e}$  with no party holding  $x_{slot,e}$  in the clear (Schoenmakers PVSS [49]).
2. MPC over the SHA-256 / SHAKE hash tree to compute the public Merkle root from secret-shared leaves without revealing the leaves (Damgård–Pastor–Smart–Zakarias SPDZ 2012, Wang–Ranellucci–Katz “Global-Scale Secure Multiparty Computation” 2017, Boyle–Gilboa–Ishai “Function Secret Sharing” 2015).

The PVSS layer (1) ships as a research-grade skeleton at `ref/go/pkg/thbs/dkg2/`: `Deal`, `Verify`, `Complaint`, `Defense`, and `Run` are wire-shape defined, with sentinel errors `ErrSkeletonOnly` and `ErrMPCRootNotImpl` preventing production consumption. The MPC-root layer (2) is open research tracked at `BLOCKERS.md:MAGNETAR-PUBLIC-DKG-1`; the candidate cost estimate is multi-second-per-signature at SHAKE-192s (consistent with the  $\sim 10^4$  hash invocations under MPC). Until both layers land, THBS is appropriate only for the TEE-attested custody path.

### A.1 THBS protocol surface

The wire surface lives at `ref/go/pkg/thbs/`:

- `thbs.DealerDKG(params, t, n, masterSeed)`  
 $\rightarrow$  (`[]PolyKeyShare`, `*ThresholdPublicKey`) — the dealer-backed setup. The dealer

derives the WOTS+ chain heads  $\mathbf{w}_{layer,tree,leaf,c}$  and FORS secret leaves  $\mathbf{f}_{tree,k,i}$  from *masterSeed* exactly as FIPS 205 KEYGEN does, Shamir-shares each element across the committee, emits one PolyKeyShare per party carrying the per-element shares, and erases *masterSeed*.

- `thbs.SignShare(share, slot, message, ctx)`  
 $\rightarrow$  *\*PartialSignature* — party  $i$ 's contribution. The party deterministically derives the message-bound randomiser  $R = \text{PRF}(\text{share.SKprf}, \text{ctx} \parallel \text{message})$  and the FIPS 205 ADRS / leaf selectors from  $R$ , then releases shares of *only* the selected WOTS+ chain heads and FORS leaves. Non-selected elements stay shared.
- `thbs.Aggregate(t, partials)`  
 $\rightarrow$  (`[]byte FIPS 205 sig`) — the aggregator. Given  $\geq t$  partial signatures with consistent slot binding, Lagrange-reconstructs the selected WOTS+ chain heads and FORS leaves, assembles the WOTS+ chain evaluations and the FORS authentication paths, and emits a wire-form FIPS 205 signature.
- Slot-binding evidence: same-slot-different-message partial signatures emit `SlotConflictEvidence` carrying the offending message pair and signatures, suitable for on-chain slashing.

**Selected-element reconstruction is the load-bearing property.** Unlike reveal-and-aggregate over a master seed (Appendix B), THBS at sign time reconstructs *only the WOTS+ chain heads and FORS leaves selected by  $R$*  — typically a few hundred elements per signature, not the full secret key. The aggregator never holds *SK.seed*, *SK.prf*, or the unselected elements. The trust posture is strictly tighter than reveal-and-aggregate; the aggregator's TCB exposure is the union of selected elements, not the full secret.

**Slot guard.** The same WOTS+ chain head used to sign two distinct messages enables WOTS+-forgery extraction by an attacker who collects both partial signatures, which is the standard hash-based one-time-signature failure mode. THBS guards against this by persisting full `PartialSignature` records keyed by  $(\text{slot}, \text{digest})$ : a second `SignShare` request with the same *slot* but a different *digest* aborts and emits `SlotConflictEvidence` verifiable post-restart. The slot counter monotonicity is enforced by the same consensus-layer invariant Pulsar uses for  $\kappa$  (cf. [28] §5).

## A.2 Construction-equivalence to single-party FIPS 205

The McGrew construction has the McGrew Class-N1-analog property: the output signature is byte-identical to the single-party FIPS 205 signature that would be produced on the same  $(\text{masterSeed}, \text{message}, \text{ctx}, \text{slot})$  tuple. This is by construction: the dealer's Shamir sharing preserves additive structure over the prime field; Lagrange reconstruction recovers the selected chain heads and leaves exactly; the WOTS+ chain evaluations are then computed identically to single-party FIPS 205. The same FIPS 205 verifier accepts the THBS output unmodified.

## A.3 Where THBS lives in the Lux stack

**M-Chain bridge custody.** The Lux M-Chain bridge custody keypairs sign withdrawal authorisations under SLH-DSA. The custody committee runs THBS v1 inside SEV-SNP-attested confidential compute, with the dealer co-located in the same attestation chain as the consensus layer. The setup ceremony is run once per key generation, with the master seed inside the enclave and the `SlotConflictEvidence` feed exposed as a public log.

**A-Chain confidential compute.** The A-Chain confidential compute layer uses THBS for the cross-shard confidential-attestation keypairs. Same attestation discipline as M-Chain bridge custody.

**NOT in scope for public-BFT consensus.** Lux P-Chain, X-Chain, C-Chain, and the white-label EVM L1/L2s (Hanzo, Zoo, Pars primary networks) run the per-validator standalone primitive of the main paper, never THBS.

## B Reveal-and-Aggregate over the SLH-DSA Scheme Seed (TEE-only)

This appendix documents the *reveal-and-aggregate* threshold construction shipped at `ref/go/pkg/magnetar/combine` under the entry point `magnetar.CombineWithSeedReconstruction`. The construction is the canonical hash-based instantiation of the “share-the-seed-and-reconstruct” pattern: the SLH-DSA scheme seed  $S$  is byte-wise Shamir-shared over  $\text{GF}(257)$  across a  $(t, n)$  committee at DKG time, and at sign time the aggregator recovers  $S$  via Lagrange interpolation in memory, invokes the single-party FIPS 205 `SIGNDETERMINISTIC`, and zeroizes the secret material before return. The output signature is byte-identical to a centralized FIPS 205 signature on the same  $(seed, \mu, ctx)$  triple.

**Trust model: TEE-only, by policy.** The aggregator process holds the reconstructed scheme seed  $S$  in memory for the duration of one `Combine` call ( $\sim 1\text{--}50$  ms wall-clock on M2 Pro at SHAKE-192s, dominated by the SLH-DSA signing latency). An adversary with arbitrary code-execution rights on the aggregator host during this window can extract  $S$  via the standard kernel-level vectors (`/proc/<pid>/mem`, `ptrace`, `coredumps`, `swap-file` leaks). This is the *aggregator-honest* caveat, and it is the same trust model the earliest Magnetar releases (v0.1, v0.2) shipped under as the headline construction. The trust caveat is not a construction-level flaw; it is intrinsic to any reveal-and-aggregate pattern over a hash-based secret. The pattern is appropriate when the aggregator host is hardware-attested confidential compute (AMD SEV-SNP with remote attestation, Intel TDX, or equivalent), in which case the trust model collapses to “the attested enclave is honest” — a defensible posture for M-Chain bridge custody and A-Chain confidential compute.

**NOT in scope for public-BFT consensus.** Public-BFT Lux validator quorums use the per-validator standalone primitive of the main paper. The reveal-and-aggregate construction is appropriate only when the aggregator host is in the TCB by deployment policy.

### B.1 Reveal-and-aggregate protocol surface

The construction is byte-exact to the entry point at `ref/go/pkg/magnetar/combine.go` and the protocol overlay under `ref/go/pkg/magnetar/dkg.go` and `ref/go/pkg/magnetar/threshold.go`.

#### DKG (three-round broadcast).

1. **Round 1 (dealer  $P_i$ ).** Sample a contribution  $c_i \in \{0, 1\}^{8 \cdot \text{SEED\_SIZE}}$  and  $2(t-1) \cdot \text{SEED\_SIZE}$  bytes of polynomial coefficient material. For each byte position  $b$ , define  $f_{b,i}(x) = c_i[b] + \sum_{j=1}^{t-1} a_{b,j,i} \cdot x^j \pmod{257}$ . For each recipient  $P_k$ , compute  $share_{i \rightarrow k}[b] = f_{b,i}(x_k) \pmod{257}$  and broadcast the per-recipient envelope.
2. **Round 2 (party  $P_i$ ).** Validate every received envelope; compute and broadcast the transcript digest  $D = \text{cSHAKE256}(\tau_{\text{DKG}}, \dots, 32\text{B})$  binding the full Round-1 record.
3. **Round 3 (party  $P_i$ ).** Verify every other party’s  $D$  matches the locally computed value; an equivocation emits identifiable abort evidence naming the offending dealer. Compute the per-party share of the master byte-sum, the master byte-sum itself (sum of dealer contributions modulo 257), and the committee root. Mix to the master seed via  $S = \text{cSHAKE256}(\text{masterByteSum} \parallel \text{committee\_root}, \dots, \text{SEED\_SIZE B})$ , derive  $(sk, pk) \leftarrow \text{SLHDSA.DERIVEKEY}$ , publish  $pk$  as the joint group public key, discard  $sk$ , zeroize  $S$ .

### Threshold sign (two-round commit-and-reveal).

1. **Round 1 (signer  $P_i$ ).** Derive a per-attempt mask  $r_i$  via cSHAKE256, XOR-mask the local share  $masked_i = share_i \oplus r_i$ , build the binding transcript  $\tau_1$ , and broadcast a 32-byte commit  $D_i = \text{cSHAKE256}(r_i \parallel masked_i \parallel \tau_1, \dots)$ .
2. **Round 2 (signer  $P_i$ ).** Validate session binding; broadcast the reveal  $r_i \parallel masked_i$ .

**Combine (aggregator).** The aggregator validates the Round-2 reveals against the Round-1 commits in constant time, recovers  $share_i = masked_i \oplus r_i$  for each signer in the quorum, runs Lagrange interpolation over GF(257) on the first  $t$  recovered shares to obtain *reconstructedByteSum*, recomputes *committee\_root* from the full **KeyShare** directory, mixes via cSHAKE256 to the reconstructed master seed  $S_{\text{rec}}$ , derives  $(sk_{\text{rec}}, pk_{\text{rec}}) \leftarrow \text{SLHDSA.DERIVEKEY}(S_{\text{rec}})$ , verifies  $pk_{\text{rec}} = pk$ , invokes either **SIGNDETERMINISTIC** (default) or **SIGNRANDOMIZED** on  $sk_{\text{rec}}$ , and zeroizes  $S_{\text{rec}}$ ,  $sk_{\text{rec}}$ , and the intermediate buffers before return.

## B.2 Byte-equality to single-party FIPS 205

**Theorem 4** (Reveal-and-aggregate byte equality). *Fix a parameter set  $MODE \in \{\text{SHAKE-192s}, \text{SHAKE-192f}, \text{SHAKE-192b}\}$  and let  $(n, t)$  be a valid threshold configuration. Let **DKG** complete honestly, producing a joint public key  $pk$  and shares  $\{\text{KEYSHARE}_j\}_{j=1}^n$ . Let  $S$  be the **DKG** master seed.*

*For any message  $\mu$ , context  $ctx$ , session identifier  $sid$ , attempt  $\kappa$ , and quorum  $Q$  with  $|Q| = t$ , if every party in  $Q$  executes the threshold-sign protocol honestly and the aggregator executes **Combine** honestly with  $\text{randomized} = \text{false}$ , then*

$$\text{COMBINE}(\dots) = \text{SLHDSA.SIGNDETERMINISTIC}(\text{SLHDSA.DERIVEKEY}(S), \mu, ctx).$$

*Proof sketch.* By inspection. Any quorum  $Q$  of size  $\geq t$  reconstructs the same *masterByteSum* from  $\{mySumShare_i\}_{i \in Q}$  via the standard Shamir Lagrange identity. The cSHAKE256 mix is deterministic, so the reconstructed seed equals the DKG master seed. FIPS 205 **DERIVEKEY** and **SIGNDETERMINISTIC** are deterministic functions of their inputs, so the output signature is byte-identical to the single-party FIPS 205 output on the same  $(S, \mu, ctx)$ .  $\square$

**Empirical pin.** `TestN1_ByteEquality_ThresholdMatchesCentralized` at `ref/go/pkg/magnetar/n1_byte` runs the threshold path end-to-end against the centralized FIPS 205 path and asserts byte-equality across  $(n, t) \in \{(3, 2), (5, 3), (7, 4)\}$ .

## B.3 Where reveal-and-aggregate lives in the Lux stack

**M-Chain bridge custody.** The reveal-and-aggregate construction is the simpler of the two TEE-only custody options; its trust model is slightly coarser than THBS (the aggregator holds the full  $S$ , not just selected elements), but it is operationally simpler to deploy and audit. `CombineWithSeedReconstruction` is the entry point.

**A-Chain confidential compute.** Same posture as M-Chain bridge custody. Pick reveal-and-aggregate when operational simplicity dominates; pick THBS (Appendix A) when the tighter TCB exposure on selected elements is preferred.

**NOT in scope for public-BFT consensus.** Public-BFT deployments use the per-validator standalone primitive of the main paper, never reveal-and-aggregate.

## B.4 Operational hardening (TEE deployment)

For any deployment running the reveal-and-aggregate construction inside a hardware-attested enclave, the `DEPLOYMENT-RUNBOOK.md` §“Aggregator hardening” enumerates the production checklist:

1. Run the aggregator in a memory-encrypted VM (AMD SEV-SNP or Intel TDX) with remote attestation; verify the attestation on every signing session.
2. Disable coredumps (`ulimit -c 0`, `kernel.core_pattern → /dev/null`).
3. Disable swap (`swapoff -a`); disable hibernation (`systemctl mask hibernate.target sleep.target`).
4. Disable ptrace (`kernel.yama.ptrace_scope = 3`); bind the aggregator under `PR_SET_DUMPABLE = 0`.
5. Fork-per-signature: `execve` a fresh image and exit immediately after `Combine` returns. This bounds the in-memory lifetime of  $S$  to a single sign call.
6. Bind under seccomp-bpf with a whitelist that forbids `ptrace`, `process_vm_readv`, and arbitrary `mmap` of `/proc/self/mem`.
7. Run on dedicated hardware: no other user-controlled processes on the aggregator host.

## References

- [1] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors, 2015.
- [2] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub. The last mile: High-assurance and high-speed cryptographic implementations. *IEEE S&P*, 2020.
- [3] Anonymous. Threshold UOV: A threshold multivariate signature. NIST MPTC submission, 2024.
- [4] Anonymous. Two-round threshold signature from algebraic one-more learning with errors. In *EUROCRYPT*. Springer, 2024.
- [5] Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. EasyCrypt: A tutorial. In *Foundations of Security Analysis and Design VII*. Springer, 2013.
- [6] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *SODA*, 2016.
- [7] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *STOC*. ACM, 1988.
- [8] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. SPHINCS+: Submission to the NIST post-quantum project. <https://sphincs.org>, 2022.
- [9] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The KECCAK reference. <https://keccak.team>, 2011.
- [10] Ward Beullens. MAYO: A multivariate quadratic signature scheme. NIST PQC additional signatures, 2023.
- [11] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In *CRYPTO*, pages 565–596. Springer, 2018.

- [12] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In *ASIACRYPT*. Springer, 2001.
- [13] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2023.
- [14] Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail / corona: Practical two-round threshold signatures from module-LWE and module-SIS. IACR ePrint 2024/1113, 2024.
- [15] Sean Bowe. BLS12-381: New zk-SNARK elliptic curve construction. <https://electriccoin.co/blog/new-snark-curve/>, 2017.
- [16] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS*. IEEE, 2001.
- [17] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. Cryptology ePrint Archive, Paper 2021/060, 2021.
- [18] Andrea Celi, Rafaël del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Efficient threshold ML-DSA. In *USENIX Security*, 2025.
- [19] Cloudflare. circl: Cloudflare interoperable reusable cryptographic library. <https://github.com/cloudflare/circl>, 2025. v1.6.3; sign/slhdsa package implements FIPS 205.
- [20] Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. RFC 9591: The flexible round-optimized schnorr threshold (FROST) protocol for two-round schnorr signatures. RFC Editor, 2024.
- [21] Andrei Cosmin and Dmitry Khovratovich. Threshold SPHINCS+: A hash-based threshold signature. In *SAC*. Springer, 2021.
- [22] Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. Two-round  $n$ -out-of- $n$  and multi-signatures and trapdoor commitment from lattices. In *PKC*. Springer, 2021.
- [23] Rafael del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Mark-Henry Servera. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. Eurocrypt, 2024.
- [24] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2018, pages 238–268, 2018.
- [25] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON: Fast-fourier lattice-based compact signatures over NTRU. <https://falcon-sign.info>, 2020.
- [26] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game. In *STOC*. ACM, 1987.
- [27] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Chapman & Hall/CRC, 3rd edition, 2020.

- [28] Zach Kelling. LP-073: Pulsar: Dynamic lattice threshold signatures for permissionless validator sets. Lux Industries; `papers/lp-073-pulsar/lp-073-pulsar.tex`, 2026.
- [29] Zach Kelling. QuasarCert soundness: Canonical formal specification, Quasar 3.0. Lux Industries; `lux/proofs/quasar-cert-soundness.tex`, 2026.
- [30] Zach Kelling and Vishnu Seesahai. LP-104: Corona / ringtail: Lattice-based post-quantum threshold signatures for blockchain consensus. Lux Industries; `papers/lux-corona-pq/lux-corona-pq.tex`, 2026.
- [31] John Kelsey, Shu jen Chang, and Ray Perlner. NIST SP 800-185: SHA-3 derived functions: cSHAKE, KMAC, TupleHash and ParallelHash. <https://csrc.nist.gov/publications/detail/sp/800-185/final>, 2016.
- [32] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In *SAC*, 2020.
- [33] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized schnorr threshold signatures. RFC 9591, 2024.
- [34] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In *CRYPTO*, 2015.
- [35] Lux Core Team. LP-020: Quasar consensus 3.0. <https://github.com/luxfi/lps/blob/main/LP-020-quasar-consensus.md>, 2026.
- [36] Lux Core Team. LP-075: BLS aggregate signatures for warp messaging. <https://github.com/luxfi/lps/blob/main/LP-075-bls-aggregate.md>, 2026.
- [37] Lux Core Team. Magnetar: Per-validator standalone FIPS 205 SLH-DSA for public-bft, with THBS and reveal-and-aggregate TEE-only forms (go canonical). <https://github.com/luxfi/magnetar>, 2026. Tag v0.5.2; canonical MAGS/MAGG wire codec and stateless VerifyBytes at commit 1c84519; per-validator standalone primary at commit 111df91; THBS v1 at commit 5100ffc.
- [38] Lux Industries. Lean 4 cryptographic proof tree (`lux-formal-verification-lean4`). <https://github.com/luxfi/proofs/tree/main/lean/Crypto>, 2026.
- [39] Lux Industries. LP-103: Lens — curve-based threshold signatures with dynamic resharing. Lux Improvement Proposal, 2026.
- [40] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. *Journal of the ACM*, 60(6):1–35, 2013.
- [41] David McGrew, Scott Fluhrer, Stefan-Lukas Gazdag, Panos Kampanakis, Michael Morton, and Bas Westerbaan. Coalition and threshold hash-based signatures. IACR ePrint 2019/793; <https://eprint.iacr.org/2019/793>, 2019. Companion IRTF draft-mcgrew-hash-sigs.
- [42] NIST. FIPS 203: Module-lattice-based key-encapsulation mechanism standard. <https://csrc.nist.gov/pubs/fips/203/final>, 2024.
- [43] NIST. FIPS 204: Module-lattice-based digital signature standard. <https://csrc.nist.gov/pubs/fips/204/final>, 2024.
- [44] NIST. FIPS 205: Stateless hash-based digital signature standard. <https://csrc.nist.gov/pubs/fips/205/final>, 2024. Final standard published August 2024.



- [45] NIST. NIST IR 8214C: Multi-party threshold cryptography schemes. NIST Internal Report (Draft), 2024.
- [46] NIST Multi-Party Threshold Cryptography Project. Hermine: Threshold lattice signatures with DKG and proactive refresh. NIST MPTC presentation, 2025.
- [47] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):1–40, 2009.
- [48] Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. Dude, is my code constant time? Design, Automation & Test in Europe, 2017.
- [49] Berry Schoenmakers. A simple publicly verifiable secret sharing scheme and its application to electronic voting. In *CRYPTO*. Springer, 1999.
- [50] Vishnu J. Seesahai and Zach Kelling. LSS: A pragmatic framework for dynamic and resilient threshold signatures with live secret resharing. Lux Industries, Inc., 2025.
- [51] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

## Reproducibility

**Source code.** [github.com/luxfi/magnetar](https://github.com/luxfi/magnetar) at tag v0.5.2 (canonical wire codec + stateless verifier commit 1c84519; per-validator standalone primary primitive 111df91; honest-aggregate transitional path 5100ffc; v0.3 Tier-A documentation shape commits 572147c through db6c252).

**Build.** `cd ref/go && GOWORK=off go build ./... && GOWORK=off go test -short ./pkg/magnetar/.` The race-detector (`-race`) is supported but skips the SLH-DSA heavy paths via the `raceEnabled` build-tag pattern; `race-on` runs cover the cheap unit tests (shamir, transcript, types, wire).

**Headline regression.** `go test -count=1 -run TestMagnetar_Wire_FIPS205Verifiable ./ref/go/pkg/magnetar/... pins` byte-identity between `ValidatorSign` output (after MAGS envelope strip) and `cloudflare/circl/sign/slhdsa.Verify` across all three parameter sets.

**Hardware tested.** Apple Silicon M2 Pro (12-core), macOS 15.4, Go 1.26.3 darwin/arm64. Linux x86\_64 reference timings will land in the v0.6 evaluation appendix.

**Dependencies.** [github.com/cloudflare/circl](https://github.com/cloudflare/circl) v1.6.3 (FIPS 205 SLH-DSA reference, BSD-3-Clause) and [golang.org/x/crypto](https://golang.org/x/crypto) v0.32.0 (cSHAKE256 over FIPS 202 / NIST SP 800-185). No private dependencies.

**Standards referenced.** FIPS 205 (SLH-DSA), FIPS 202 (SHA-3 / SHAKE / cSHAKE), NIST SP 800-185 (KMAC / TupleHash / cSHAKE customisation), NIST IR 8214C [45] (Multi-Party Threshold Cryptography scope).

**Companion documents.** `SPEC.md` (normative construction), `DEPLOYMENT-RUNBOOK.md` (public-BFT default + TEE-only threshold paths), `BLOCKERS.md` (open gates), `CHANGELOG.md` (release-by-release framing), `SUBMISSION.md` (NIST MPTC scaffold), `TRUSTED-COMPUTING-BASE.md` (TCB enumeration), `PROOF-CLAIMS.md` (proof-roadmap inventory), `AXIOM-INVENTORY.md` (assumed-axiom enumeration), `FIPS-TRACEABILITY.md` (line-by-line conformance trace).

**Contact.** [security@lux.network](mailto:security@lux.network).